Swarm Intell (2011) 5:183–223
DOI 10.1007/s11721-011-0063-y

Slime mold inspired routing protocols for wireless sensor networks

Ke Li · Claudio E. Torres · Kyle Thomas · Louis F. Rossi · Chien-Chung Shen

Received: 1 November 2010 / Accepted: 7 October 2011 / Published online: 10 November 2011
© Springer Science + Business Media, LLC 2011

Abstract Many biological systems are composed of unreliable components which self-organize effectively into systems that achieve a balance between efficiency and robustness. One such example is the true slime mold *Physarum polycephalum* which is an amoeba-like organism that seeks and connects food sources and efficiently distributes nutrients throughout its cell body. The distribution of nutrients is accomplished by a self-assembled resource distribution network of small tubes with varying diameter which can evolve with changing environmental conditions without any global control. In this paper, we exploit two different mechanisms of the slime mold's tubular network formation process via laboratory experiments and mathematical behavior modeling to design two corresponding localized routing

Preliminary versions of this paper appeared in the following two conferences: (1) 2nd IEEE International Conference on Self-Adaptive and Self-Organizing Systems (SASO), Venice, Italy, October 20–24, 2008, and (2) 7th International Conference on Swarm intelligence (ANTS), Brussels, Belgium, September 8–10, 2010.

Electronic supplementary material The online version of this article (doi:10.1007/s11721-011-0063-y) contains supplementary material, which is available to authorized users.

K. Li · C.-C. Shen (✉)
Department of Computer and Information Sciences, University of Delaware, Newark, DE, USA
e-mail: cshen@cis.udel.edu

K. Li
e-mail: kli@cis.udel.edu

C.E. Torres · L.F. Rossi
Department of Mathematical Sciences, University of Delaware, Newark, DE, USA

C.E. Torres
e-mail: torres@math.udel.edu

L.F. Rossi
e-mail: rossi@math.udel.edu

K. Thomas
Department of Chemical Engineering, University of Delaware, Newark, DE, USA
e-mail: kcthomas@udel.edu

protocols for wireless sensor networks (WSNs) that take both efficiency and robustness into account. In the first mechanism of *path growth*, slime mold explores its immediate surroundings to discover and connect new food sources during its growth cycle. We adapt this mechanism for a path growth routing protocol by treating data sources and sinks as *singular potentials* to establish routes from the sinks to all the data sources. The second mechanism of *path evolution* is the temporal evolution of existing tubes through nonlinear feedback in order to distribute nutrients efficiently throughout the organism. Specifically, the diameters of tubes carrying large fluxes of nutrients grow to expand their capacities, and tubes that are not used decline and disappear entirely. We adapt the tube dynamics of the slime mold for a path evolution routing protocol. In our protocol, we identify one key adaptation parameter to adjust the tradeoff between efficiency and robustness of network routes. Through extensive realistic network simulations and ideal closed form or numerical computations, we validate the effectiveness of both protocols, as well as the efficiency and robustness of the resulting network connectivity.

Keywords Slime mold · Routing protocol · Wireless sensor network · Simulation stability analysis

1 Introduction

In wireless sensor networks (WSNs), routing protocols determine multi-hop paths between sensor nodes and sink node(s) to facilitate data gathering. There exist two performance objectives for such paths—*efficiency*, where it is desirable to forward data from a sensor to a sink over a minimum number of hops, and *robustness*, where it is desirable to have redundant paths between any sensor-sink pair. However, the two objectives are competing as efficiency favors single minimum-hop paths while robustness prefers multiple redundant paths.

While centralized schemes that rely upon global coordination and optimization may provide optimal tradeoff solutions to balance efficiency and robustness, they do not scale or adapt well in large and dynamic networks like WSNs. As a result, many decentralized solutions have been proposed to address these issues. Fang et al. developed a landmark-based scheme called GLIDER (Fang et al. 2005) for efficient point-to-point routing in WSNs, where global topology is discovered and partitioned into cluster-like tiles. Greedy routing is used inside tiles based on nodes' local coordinates derived from hop distances to nearby landmarks. However, GLIDER is not applicable to converge-cast (many-to-one) applications. LEACH (Heinzelman et al. 2000) and HIT (Culpepper et al. 2004) are cluster-based protocols to address energy-efficiency in data collection applications. All sensors are organized into clusters. Each elected cluster head acquires data from sensors within its own cluster either directly (Heinzelman et al. 2000) or indirectly (Culpepper et al. 2004), performs data aggregation, and transmits fused data directly to the sink, which allows most nodes to transmit in small distances and to reduce the amount of data sent in the network to save battery energy. Directed diffusion (Intanagonwiwat et al. 2003) is an application-aware paradigm that achieves efficiency by data aggregation at intermediate nodes. The sink first propagates *interests* (data requests) throughout the network, establishing *gradients* to 'draw' events. Sensor sources then send event data back via empirically good and reinforced reverse paths. More recently, Saleem et al. (2010) provide an extensive survey of swarm intelligence- (SI-) based WSN routing protocols inspired mostly by the foraging behaviors of ant and bee colonies. Using decentralized controls and self-organization, these protocols rely on the release of many mobile agents (e.g., ants or scouts) to collectively explore and

converge to the shortest path. Although multiple paths could be taken to achieve robustness, there is no easy solution to balance that with efficiency in a decentralized manner. Ducatelle et al. (2010) further investigate the principles of SI for adaptive routing in telecommunications networks, and define Ant Colony Routing as a general framework for SI-based routing protocol design. They also present an extensive review of SI-based unicast routing protocols in both wired and wireless networks.

To create efficient and robust routes in WSNs while relying solely upon local information, we seek mechanisms from similar problems in nature and adapt them to suit the challenges of WSNs. Through natural selection, biological systems often achieve a balance between efficiency and robustness via self-organization without any global coordination or information. One such system is the true slime mold *Physarum polycephalum* which self-assembles its nutrient distribution networks in response to dynamic environmental conditions—distinct from circulatory systems hard-wired for a particular physiology.

Slime mold *Physarum polycephalum* is a multi-nuclear, single-celled organism that can grow to tens of centimeters in body size, allowing it to be studied and manipulated with modest laboratory facilities. During the plasmodium phase, the organism spans out searching for organic nutrients in the environment, which are transported through self-assembled tubular channels that form a dynamic network throughout the cell body. These tubes also act as *pseudopodia* which allows the slime mold to navigate around its environment (Stewart 1964). At times, the slime mold disassembles and reassembles these tubes to connect new food sources into its network and sustain itself by distributing nutrients through the organism. Fitness requires that resources be discovered in the environment and distributed efficiently throughout the organism. Thus *Physarum polycephalum* is ideal for studying the function and dynamics of natural adaptive resource distribution networks (Ben-Jacob and Cohen 1998). Experimental observations have shown that slime mold will connect multiple separate nutrient sources into a variety of final configurations including Steiner trees, spanning trees, other more redundant structures like cycles, and combinations of these patterns (Nakagaki et al. 2004a). Often, the tubes of a slime mold will be arranged in a geometry that balances both efficiency (keeping the total tube length short) and robustness (having multiple paths in case a tube is severed). These observations are entirely consistent with localized algorithms lacking complete global information to build resource distribution networks considering both performance metrics.

Inspired by two different laboratory experiments, this paper exploits two different tubular network formation processes of slime mold via mathematical behavior modeling to design two corresponding localized routing protocols for WSNs that balance efficiency and robustness. In the first mechanism of *path growth*, slime mold explores its immediate surroundings to discover and connect new food sources during its growth cycles. The path growth mechanism is adapted for WSN routing by treating data sources and sinks as *singular potentials* to establish routes from the sinks to all sensor sources. The second mechanism of *path evolution* is the temporal evolution (or selection) of existing tubes through nonlinear feedback so as to distribute nutrients efficiently throughout the organism. Specifically, the diameters of tubes carrying large fluxes of nutrients grow to expand their capacities, while tubes that are not used may decline and disappear entirely. We apply this tube dynamics to WSNs routing with one key parameter to adjust the tradeoff between efficiency and robustness of routing paths. In a recent article on slime mold-inspired adaptive network design, Tero et al. (2010) also independently acknowledged the suitability of this type of model for WSNs.

The remainder of this paper is organized as follows. In the next section, we describe two mathematical models which characterize the path growth and the path evolution behaviors of the slime mold's tubular networks, respectively. In Sect. 3, we describe the adaptations of

the two models to design two localized routing protocols for WSNs. In Sect. 4, we evaluate the designed protocols via realistic network simulation, compare the simulation results with the ideal exact or numerical results, measure the efficiency and robustness of resulting routes in quantitative metrics, and discuss the effects of different parameters on these protocols. In Sect. 5, we summarize our results and describes future research directions.

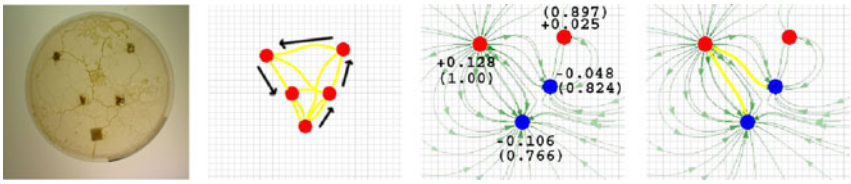
2 Mathematical models

While any living system is complex by necessity, often its specific process is dominated by a small number of primitive mechanisms. Considerable research has focused on observed behavioral manifestations of slime mold in laboratory experiments, including searching for food, avoiding danger, and determining the shortest paths through mazes under certain circumstances (Nakagaki et al. 2000, 2001, 2004a, 2004b). Also a variety of modeling approaches have been applied to understanding slime mold. Several investigators have derived models from physical principles or developed phenomenological models based on laboratory observations. Kobayashi modeled oscillations in the protoplasm using a phase field model (Kobayashi et al. 2006). Hofer and Maini modeled and analyzed the searching behavior of pseudopodia (Hofer and Maini 1997). Gunji et al. proposed a cellular model (Gunji et al. 2008), and Tero et al. have studied a tube flow model using Poiseuille flow coupled to an adaptation term (Tero et al. 2007).

We investigate two behavioral patterns of slime mold, and adapt these models to design practical routing protocols for WSNs. These two different mathematical models are inspired and validated in laboratory experiments. The first model is based on the initial growth phase of slime mold as the organism grows to occupy its domain, a petri dish in this case. During this phase, the organism will self-assemble a network of tubes that will connect food sources. In a laboratory setting, one can place slime mold in the center of a petri dish with an agar foundation along with nutrients in the form of oatmeal flakes. Several other food sources are placed at different distances from the slime mold. The slime mold will respond through chemotaxis and grow toward the distant food sources, building a network of tubes as it evolves. Based on the observed formation of the slime mold's tubular network connecting all the food sources, we propose a path growth model using singular potentials and gradient descent. The second model is based on the evolution of existing networks once the slime occupies the entire domain. In the maze solving experiment conducted by Nakagaki et al. (2000), the slime mold, initially filling everywhere in the maze, changes its shape to connect food sources at both the entrance and the exit of the maze. We adapt the tube dynamics flow model of Tero et al. (2007) for WSNs routing with single or multiple sinks as well as multi-path data forwarding.

2.1 Singular potential path growth model

Inspired by our own laboratory experiments, we propose a mathematical model to capture a key mechanism of the slime mold's initial tubular network formation process. Our model is based on slime mold's chemotactic reactions to chemical stimuli of food sources in the environment. Chemotactic response (Adler and Tso 1974) is common in many biological settings like bacteria and single-cell organisms, where the movements of a living organism are directly relative to a local chemical gradient. Notice that there is no centralized control in slime mold, and the assembly of the tubular networks is a decentralized response to local environmental cues. To mimic the dominant behavior of connecting one food source to



(a) (b) (c) (d)

Fig. 1 (**a**) A laboratory slime mold experiment showing the final tubular network configuration connecting five food sources, with the plasmodium initially placed with the bottom food. (**b**) One possible solution using the singular potential model. Notice that other solutions are possible corresponding to different orderings in how food is discovered. In this case, we choose the order in which the plasmodium discovered the food in the laboratory experiment, and compute the amount of food remaining at each food source, ψ and the resulting network paths based on this prescribed order. One of the incremental moves is shown in (**c**) and (**d**). Specifically, (**c**) shows each food node's relative nutrient value (after subtracting the mean) and absolute value (within parentheses). In (**d**), new paths are created following the minimum field lines

another, we model the chemical environment (Conolly et al. 1997) as a scalar field using *singular potentials* located at each food source. A *singular potential* or *fundamental solution* ψ generates a divergence-free ($\nabla^2\psi$) vector field everywhere except at its location. Thus, we are collapsing the food source to a point. A fundamental solution (John 1986) ψ satisfies

$$\nabla^2\psi = \delta(\mathbf{x} - \mathbf{x}_i), \tag{1}$$

where $\mathbf{x}_i$ is the position vector of the ith food source and δ is the *Dirac* delta distribution. Since (1) is linear, we can represent multiple food sources as a sum of the fundamental solutions. Solutions to this equation are well studied, which are logarithmic in two-dimensional domains. In our path growth model, tubes will self-assemble sequentially following $\nabla\psi$ (the gradient of ψ), referred to as a *gradient descent*. A food source will be consumed once it is connected to the network, which in return will affect ψ.

Using the above formulation, we design the following algorithm to compute the growth of slime mold's tubular network.

1. Initialize slime mold at one or more food sources.
2. The slime mold would then stream toward a new food source following a gradient descent. The total field φ is the summed contribution or *linear superposition* of the fundamental solutions (1). This total field has the form

$$\varphi = \sum_{i=1}^{N} (f_i - \bar{f}) \ln(|\mathbf{x} - \mathbf{x_i}|^2), \tag{2}$$

   where N is the number of discovered food sources, f_i is the amount of nutrient remaining at the ith food source and $\bar{f}$ is the mean of all f_i. The initial direction of the search may be random. However, once the initial step is taken, the path is determined by the gradient of φ until reaching a new food source.
3. When slime mold reaches a new food source, the new food source is connected to the network. Nutrient values of connected food sources decay.
4. If there exist unconnected food sources, go to Step 2. Terminate otherwise.

In this algorithm, the slime mold acquires new food sources incrementally using local interactions based on information of discovered food sources, which is suitable for route

establishment in WSNs. The use of singular potentials is similar to the description of electric fields and fluid flows using potentials. If f_i's represented electric charges instead of nutrient values, $\nabla\varphi$ would denote electric field lines. If these food sources were replaced by fluid sources and sinks with specified mass fluxes, φ would be the stream function with lines of constant φ corresponding to material paths of the fluid flow.

To validate our model, we perform a series of laboratory experiments. Slime mold is grown in petri dishes on agar. Nutrient sources of blended oats in gelatin are placed at points of interest on the agar to observe the self-assembly of slime mold's tubular networks. We found that one cubic centimeter of nutrient blend would be consumed in about two days. We also assume that nutrients will be consumed linearly over time. Figure 1 compares the network structures of a laboratory experiment and the corresponding solution from our model. As observed, the tube formations of slime mold could be reproduced using singular potentials with corresponding estimates of the nutrient content available at each food source, which validates our model.

2.2 Tube dynamics path evolution model

Another key mechanism in slime mold is the dynamic evolution of its tubular network that enables efficient and robust nutrients distribution throughout its body. Specifically, self-assembled tubes evolve over time without any global control such that tubes with larger nutrient flows grow thicker, while those with smaller flows degenerate and eventually disappear.

Inspired by their maze solving experiments (Nakagaki et al. 2000), Tero et al. (2007) proposed a mathematical model to capture the evolution of tube capacities as well as network topology in an existing maze network via a dynamic feedback system. Specifically, the tubular network is represented by a graph of nodes and edges, with each node representing either a food source (at the entrance or the exit) or a junction in the maze, and each edge representing a tube between two nodes. Along each edge, the flow (or flux) of nutrients from node i to node j is denoted as q_{ij}, which is directional with $q_{ij} = -q_{ji}$. Flow through the network is driven by the different pressure p_i at each node. D_{ij} denotes the conductivity (or "diameter") of the tube, which is assumed to be symmetric (i.e., $D_{ij} = D_{ji}$). Variables of this dynamic feedback system evolve over time as follows:

$$q_{ij} = \frac{D_{ij}}{L_{ij}}(p_i - p_j), \tag{3a}$$

$$\sum_{j \in N_i} q_{ij} = m_i, \tag{3b}$$

$$\frac{dD_{ij}}{dt} = g\big(|q_{ij}|\big) - rD_{ij}. \tag{3c}$$

In this dynamic system, (3a) defines the nutrient flux through a tube as approximately a *Poiseuille* flow, where L_{ij} denotes the length of the tube. The continuity equation (3b), analogous to the *Kirchhoff* law, enforces conservation of nutrient flux at each node, where the source value m_i denotes the net flux *out* of node i, and the neighbor set N_i includes nodes with direct edges to node i. Last, the ordinary differential equation (ODE) (3c) describes the adaptation of tube size with feedback controls, where function $g(q)$ denotes the growth rate of the tube size in response to the nutrient flow, $q > 0$, through the tube, and r indicates the linear decay rate of tube size.

In our study, we represent $g(q)$ slightly differently from the original model (Tero et al. 2007) as the following sigmoidal function:

$$g(q) = r D_{\max} \frac{a|q|^{\mu}}{1 + a|q|^{\mu}}, \tag{4}$$

where $D_{\max}$ sets the upper bound to tube size D_{ij}'s. Note that the exponent $\mu > 0$ plays a critical role in feedback regulation of the model, which can adjust route selection between efficient single paths ($\mu > 1$) and robust multiple paths ($0 < \mu < 1$), as will be discussed in Sect. 4.2. The parameter a describes the shape of the sigmoid and has little impact on routes selections.

In the original model (Tero et al. 2007), the net outflux of food nodes are designated to be $m_i = +1$ at the entrance and $m_i = -1$ at the exit so as to drive flow through the maze, and $m_i = 0$ for all the other nodes (which are junction nodes within the maze). Although it may seem arbitrary to assign identical nutrient sources as either $m_i = +1$ or $m_i = -1$ in slime mold maze solving, this distinction maps well to a WSN which includes sensors that are data sources and data sinks to collect sources data. However, the pressures (p_i) are indeterminate when the sources (m_i) are assigned to every node because the linear system (3a) and (3b) can only determine pressure *differences* between nodes. We resolve this issue by specifying an invariant pressure *datum* of zero rather than a negative net outflux at the exit node of the maze, as a baseline (minimum pressure) to derive pressure values for all other nodes.

In comparison to Ant Colony Optimization (ACO) (Dorigo and Stutzle 2004), the tube evolution model of slime mold also includes dynamic positive and negative feedback mechanism in (3c). The tube diameter in the slime mold model maps to pheromone values in ACO. Both the tube diameter and pheromone levels decay linearly. Both tube diameter and pheromone levels are reinforced through nonlinear feedback. The key distinction is that pheromone levels are reinforced locally through ant traffic in ACO. In contrast, the slime model permits nearly simultaneous coordination through flow continuity in (3a) and (3b), which provides both the advantage that more information is available, and the disadvantage that pressure values have to be determined iteratively.

We develop our slime mold algorithm knowing that unique challenges arise when solving system (3) across a set of networked nodes. In particular, we assume that node i only has reliable information about its own local variables p_i, D_{ij} and m_i. Information about pressure at neighboring nodes, p_j for $j \in N_i$, needs to be updated through communication. However, we can determine the local pressure if values of D_{ij}, p_j for $j \in N_i$ or m_i change: By substituting the flux (3a) into the Kirchhoff law (3b) and solving the resulting linear equation system for p_i, we have

$$p_i^{\text{update}} = \frac{m_i + \sum_{j \in N_i} \frac{D_{ij}}{L_{ij}} p_j}{\sum_{j \in N_i} \frac{D_{ij}}{L_{ij}}}. \tag{5}$$

Once the pressure is determined, we need to advance D_{ij} in time for a discrete time step Δt by solving the ODE (3c) numerically. We use the first-order implicit scheme proposed in Tero et al. (2007):

$$D_{ij}^{(n+1)} = \frac{D_{ij}^{(n)} + g(|q_{ij}^{(n)}|)\Delta t}{1 + r\Delta t}, \tag{6}$$

where the fluxes $q_{ij}^{(n)}$'s are determined by (3a) using the solved pressures p_i, Δt is the duration of one discrete time step, and $D_{ij}^{(n)}$ is the value of D_{ij} at the nth step.

This suggests the following local algorithm for solving system (3) on node i.

1. Update the local pressure, p_i, using (5).
2. Update the tube diameters, D_{ij} for all $j \in N_i$, using (6) and marching forward one time step Δt.
3. Repeat steps 1 and 2 until a communication event occurs which provides updated information about the pressures p_j on neighboring nodes.
4. Broadcast the local pressure p_i to neighbors.
5. Go to step 1.

If we hold the D_{ij}'s and m_i's constant and assume synchronous, ideal communication across all nodes, the proposed algorithm would reduce to the famous Jacobi algorithm for solving linear systems. Essentially, this would involve updating the local pressure and then sharing the results with neighbors and iterating. Similar to the Jacobi algorithm for solving diagonally dominant linear systems, the above method of solving the Kirchhoff law (3b) in terms of pressure is *translation invariant*, meaning that one can take a pressure solution set and add the same constant to all pressures to obtain another solution to the system. Therefore, we fix the pressure to be zero without considering the Kirchhoff's law at a single datum node (i.e., the maze exit).

Our scheme can be executed cooperatively in a decentralized manner at each node, except that the single datum node does not solve for its own pressure. Specifically, each node i locally maintains the values of its own pressure p_i, and link diameter D_{ij} and pressure p_j for each neighbor $j \in N_i$. During the local computation cycle, node i solves for p_i (except for the exit node) and marches its D_{ij}'s forward in time for all incident tubes for many rounds based on local information. At a fixed time interval, node i enters the synchronization cycle to exchange its latest computed pressure value with neighbor nodes $j \in N_i$, which allows globally coupled information to propagate across the network. When node i has received new pressure updates from its neighbors, node i will start over with a new cycle of local computation (using received neighbor updates) followed by the next synchronization. In the next section, we will provide a detailed description of how this algorithm can be implemented asynchronously across realistic nodes.

3 Protocol descriptions

The described mathematical models, although capturing the essence of slime mold's behavior, cannot directly function as networking protocols due to their direct access to global information without incurring any communication overhead. We adapt both mathematical models to design two corresponding localized routing protocols for WSNs that consider both efficiency and robustness.

In the first protocol, termed "path growth" protocol, the formation of routing paths starts from the data sink and grows outwards exponentially to establish connectivity to all the data sources. The protocol may also be termed "data seeking," as the sink initiates data explorations to seek and acquire all data sources into the network. Accordingly, the routing table (based on the *connection table*, to be described later) at each node starts empty (with no entry) and grows by adding new next hop(s) leading to the sink. In the second protocol, termed "path evolution" protocol, a WSN starts as a connected graph with all the possible paths (from all existent communication links), and then evolves iteratively to some selected routing paths (from a chosen subset of links). The protocol may also be termed "data sinking," as the data sink attracts data flows from all the food sources toward itself by maintaining a minimum pressure baseline and the global data flow continuity. Accordingly, the routing

table (based on the *neighbor table*, to be described later) at each node starts full (with entries to all the neighbors) and evolves to include only a selected subset of neighbors with associated flow distribution, which may be used for multi-path data forwarding.

In terms of the amount of information needed, the path growth protocol is close to link-state (LS) routing, since data sources information within *FOOD_RANGE* hops is maintained (by food discovery packets) at each node to determine the direction of exploration and each node is aware of its own location. In contrast, the path evolution protocol is close to distance-vector (DV) routing, because only 1-hop neighbor information is maintained (by 1-hop neighbor update packets) for iterative local computations and no location information is needed. In terms of convergence speed, the LS-like path growth protocol is self-terminating—it stops when all data sources are connected. Moreover, the LS-like path growth protocol terminates faster because data sources are connected at an exponential rate. In comparison, the DV-like path evolution protocol may converge slowly due to iterative computations.

3.1 Path growth (data seeking) protocol

There are a number of interesting analogies between the slime mold growth/seeking behaviors and WSNs routing schemes, so that features in the singular potential model of slime mold are readily applicable to WSNs routing. In the biological context, slime mold tries to construct an optimal tubular network connecting itself to all the food sources based on local interactions. In the context of WSNs, the localized routing protocol aims to establish optimal network routes between the data sink and all the data sources (sensor nodes with data). Intuitively, we map the food sources with nutrient contents in the slime mold to sensors that generate data in WSNs. The biological system can respond locally to a nutrient sources because nutrient sources diffuse efficiently over short distances. The chemotactic response of slime mold can be translated to the *data seeking* mechanism in the localized networking protocol. As the nutrient-seeking slime mold stretches out to reach and consume nutrients, the data sink in the localized protocol sends out explorer packets to connect and off-load data sources via single or multiple hops. At each food source that slime mold has reached, the part of the slime mold body residing there could continue extending out for new food sources. Similarly, in the routing protocol, once a data source is connected to the sink, the data source itself may also start exploring for other data sources.

Nevertheless, the model of the biological system is not entirely isomorphic to the WSNs scenarios. Specifically, we need to adapt the following two features in the singular potential model to WSNs. First, diffusive processes in biological systems interact instantaneously over arbitrary distances (though with exponentially weak interactions), whereas wireless communications have only finite transmission radius. Accordingly, we sum the potentials over source nodes only within the specified hops of transmission range. Information on neighbors and nearby data sources can be obtained by exchanging control messages locally within one or few hops. Second, the biological system builds tubes in a continuous domain whereas WSNs routing must use discrete relay nodes. Consequently, the gradient ascent is performed over a discrete mesh network. Rather than ascending in any direction, we proceed to a relay node with position vector most closely aligned with the gradient vector.

In the localized protocol, we assume that each node is aware of its own position (e.g., *Cartesian* coordinates), which would be used to calculate the displacement vector from itself to a neighbor node or a nearby data source (in (2)). For simplicity, we assume an identical transmission radius for every node in the WSNs, so that each communication link is bi-directional. We also assume a single data sink in WSNs. It is trivial for the protocol to work

with multiple sinks, as each sink will independently explore and connect data sources locally with the same data seeking mechanism. In the following description, food, food node, food source, and data source are used interchangeably in the context of WSNs routing.

The core mechanism of the data seeking protocol is food exploration via two types of control packet: FORWARD REQUEST and BACKWARD REPLY. Only food nodes and the sink are eligible for the origins or destinations of these exploration packets, where the origin of a request packet is called a *requester*, and the destination of the request is called a *replier* who also becomes the origin of its own request packet. When a neither-food-nor-sink node receives an exploration packet addressed to itself, it becomes a *forwarder* to relay the packet to a next hop node based on either gradient ascent of the singular potential (for FORWARD REQUEST) or source route (for BACKWARD REPLY).

The network starts with the sink as the only connected node, and grows outwards exponentially to establish connections to all food nodes. As always being a requester, the sink initiates the first FORWARD REQUEST and selects the next hop based on a *goodness* heuristic, heading toward a nearby food. When a food receives a fresh (unseen before) FORWARD REQUEST addressed to itself, the food echoes with a BACKWARD REPLY via the reverse exploration path, both to acknowledge the requester and to announce to others that it is connected. Only for a first-time connected food,[1] it can start one exploration by initiating a request packet. For a food connected more than once (including the sink), it has to wait until its current request is echoed by a corresponding BACKWARD REPLY in order to start a new round of exploration. Note that this *stop-and-wait* behavior also serves as the termination mechanism of the protocol. Once all food nodes are successfully connected, the gradient of the singular potential becomes zero vector ($\mathbf{0}$). Thus all request packets on the fly will be discarded due to indeterminate next hops. Therefore, the protocol simply terminates when no more exploration packet may be sent out. We describe the data seeking protocol in detail in the following.

3.1.1 Local data structures

Each node i has an associated node type ($nodeType_i$), food value ($foodVal_i$) and *Cartesian* coordinates ($xCoord_i, yCoord_i$). The value of $nodeType_i$ could be either *SLIME* (data sink, denoted as *sink*), *FOOD* (data source, i.e., sensor with data input, denoted as *food*), or *OTHER* (neither of the above, denoted as *non-food*). The $foodVal_i$ specifies the amount of data in data source i, which will be reset to zero once node i is connected as data will be off-loaded to the sink. Food value is always zero for the data sink, since it is always connected as a *zero-food*. Food value for non-food nodes is undefined and ignored. Both food values and coordinates of the food or sink nodes are used in the gradient ascent of singular potential model during food exploration. In addition, the following local data structures are also maintained at each node i:

– *Neighbor table* $nTab_i$: $nTab_i(j)$ represents the *neighbor table* entry corresponding to neighbor j of node i, which contains the polar angle of neighbor j in the polar coordinate system with node i as the pole. Note that node i's *neighbor table* specifies all nodes within i's transmission range so that i may directly communicate with them. Our protocol relies on the initial 1-hop HELLO packet from every node, which includes the sender's coordinates, to allow nodes to learn about their neighbors and to fill their *neighbor tables*. Due to the broadcast nature of wireless communications, other control packets

[1] A food node could possibly be connected multiple times from different requesters, which introduces redundant paths to improve robustness similarly to the slime mold tubular networks.

Fig. 2 ADVERTISEMENT packet format

foodId	*foodVal*	*x*	*y*	*hopCount*

(e.g., food advertisement and food explore) may also serve the role of neighbor discover by piggybacking coordinates information.

- *Connection table connTab$_i$*: *connTab$_i$* denotes the *connection table* that includes node i's *parents* (uplinks to the sink) and *children* (downlinks away from the sink) nodes, as the results of the computed network connections through food exploration. It is clear that both *parents* and *children* of node i belong to *nTab$_i$*. Note that the *connection table* may serve as the routing table for data gathering operations in WSNs, with *parents* indicating the next hop(s) leading to the sink.
- *Nearby-food table nfTab$_i$*: *nfTab$_i$*(j) denotes the *nearby-food table* entry which keeps track of a nearby food node j within *FOOD_RANGE* hops from node i. Specifically, the entry *nfTab$_i$*(j) is of the form $\langle f, d_x, d_y, d_h, n_h\rangle$, where f is food j's value, (d_x, d_y) represents the displacement vector from node i to food j, d_h is food j's hop count distance from node i, and n_h denotes the neighbor of node i through which (as the next hop) a request packet may reach food j. The *nearby-food table* is the basis for determining next hops during food exploration, by means of either the gradient ascent or the goodness heuristic.
- *Step-one-choice table socTab$_i$*: *socTab$_i$*(j) denotes the *step-one-choice table* entry representing the goodness of choosing neighbor j as the next hop during food exploration. This table is derived from and synchronized (and hence consistent) with the *nearby-food table*. In the *step-one-choice table*, neighbor j, selected as the next hop from node i, is retrieved from the next hop field n_h in node i's *nearby-food table*. The corresponding goodness heuristic, as will be defined in (7), reflects a tradeoff between the amount of food value and the hop distance of all food nodes that could be reached through neighbor j (i.e., use j as the next hop)—the larger food value and smaller hop distance through j, the better goodness value of neighbor j. Note that this heuristic metric supplements the method of gradient ascent in determining the next hop for a request packet, especially from a food node.
- Local sequence number *seqNo$_i$*: initialized as zero, which keeps track of the sequence number of the latest FORWARD REQUEST packet generated and sent out when node i is either a food node or the sink node.

3.1.2 Food discovery

In order to perform gradient ascent in the singular potential model, each node needs to know the existence of its nearby food nodes within *FOOD_RANGE* hops from itself. This information is obtained by the propagation of an ADVERTISEMENT packet from each food or sink node, who broadcasts its own ADVERTISEMENT packet at a randomly chosen time between zero and *AD_TIMER* after its initialization. The ADVERTISEMENT packet, as shown in Fig. 2, contains the sender's id (*foodId*), food value (*foodVal*), and *Cartesian* coordinates (x, y), together with a hop count (*hopCount*) field initially set to zero to record the number of hops traveled so far by this packet.

Upon receiving an ADVERTISEMENT, each node i discards the packet if it has seen an advertisement from the same originator before, since only one such packet is allowed for each food or sink. Algorithm 1 presents the pseudo code of how node i processes an ADVERTISEMENT. If this is a new advertisement, node i increments the packet's *hopCount* field by one, inserts a new entry in its *nearby-food table* based on the information from the

Algorithm 1 Node i processing an ADVERTISEMENT

Input:
$ad \leftarrow$ incoming ADVERTISEMENT
$prev \leftarrow$ the node from which ad was received

Begin:
$j \leftarrow ad.foodId$

if $j \neq i$ AND $j \notin nfTab_i$ **then**
  Increase hop count in the ad packet:
    $ad.hopCount \leftarrow ad.hopCount + 1$

  Add an entry of j to node i's *nearby-food* table:
    $nfTab_i(j).f \leftarrow ad.foodVal$
    $nfTab_i(j).d_x \leftarrow ad.x - xCoord_i$
    $nfTab_i(j).d_y \leftarrow ad.y - yCoord_i$
    $nfTab_i(j).d_h \leftarrow ad.hopCount$
    $nfTab_i(j).n_h \leftarrow prev$

  Compute $goodness_i(b)$ for node $b = nfTab_i(j).n_h$, as the ratio of food value over food's hop distance from i via b:

$$goodness_i(b) \leftarrow \frac{nfTab_i(j).f}{nfTab_i(j).d_h} \tag{7}$$

  Increase goodness in node i's *step-one-choice* table:
    $socTab_i(b) \leftarrow socTab_i(b) + goodness_i(b)$

  if $ad.hopCount < FOOD_RANGE$ **then**
    Rebroadcast ad
  end if
end if

packet, and increases the corresponding goodness value in its *step-one-choice table*. The heuristic value $goodness_i(b)$ of node b at node i reflects the desirability of node i choosing node b as the next hop for a FORWARD REQUEST. Equation (7) defines the *goodness* of neighbor b as the ratio of food value over food's hop distance from node i via neighbor b. Finally, node i rebroadcasts the ADVERTISEMENT only if the *hopCount* value is less than *FOOD_RANGE*.

3.1.3 Forward explore request

At *REQUEST_TIMER* when information on nearby food within *FOOD_RANGE* hops (as well as 1-hop neighbors) has been collected at each node, the sink becomes the first requester and initiates a FORWARD REQUEST packet to grow a path from itself connecting data sources via food exploration. Once connected, each food also becomes a requester, eligible to send its own request packets. A FORWARD REQUEST initiated by requester i, whose format is shown in Fig. 3(a), contains the following fields:

Fig. 3 Food explore packet formats: (**a**) FORWARD REQUEST and (**b**) BACKWARD REPLY

seqNo	*x*	*y*	*exLimit*	*visitedNodes...*

(a)

seqNo	*replier*	*x*	*y*	*visitedNodes...*

(b)

- *seqNo*: the sequence number of the FORWARD REQUEST packet, which, together with the requester id i stored as the first entry in *visitedNodes*, uniquely identifies the request packet.
- x, y: the *Cartesian* coordinates of requester i. Note that the coordinates, together with the implicit food id (requester i) and food value (zero), allow the explore packet to serve also as an ADVERTISEMENT packet for food information update, as shown in lines 4 and 5 of Algorithm 3 described below.
- *exLimit*: the maximum number of hop counts the request packet is allowed to explore before reaching a food node. Initially set to *EXPLORE_LIMIT*, it is decremented each time the request packet travels to the next hop.
- *visitedNodes*: the ordered list of nodes visited by the FORWARD REQUEST, initially set to $\langle i \rangle$. Notice that the first entry of the list is always the originating requester id i, and the last entry is always the next hop id.

Algorithm 2 Procedure *SendRequest*() executed by node i

1: **Begin:**
2: Increase node i's local sequence number for request:
$seqNo_i \leftarrow seqNo_i + 1$
3: Create a new FORWARD REQUEST packet *req*:
$req.seqNo \leftarrow seqNo_i$
$req.x \leftarrow xCoord_i$
$req.y \leftarrow yCoord_i$
$req.exLimit \leftarrow EXPLORE_LIMIT - 1$
$req.visitedNodes[0] \leftarrow i$
4: Determine the first jump based on *goodness*:

$$firstJump = \underset{j \in socTab_i \setminus connTab_i}{\operatorname{argmax}} \; socTab_i(j) \tag{8}$$

5: Append *firstJump* to *req.visitedNodes*
and send out *req*

A requester node i (either the sink or a food) creates its request packet and finds a next hop for the packet to explore, by invoking the procedure *SendRequest*() described in Algorithm 2. Requester i increments its local $seqNo_i$ by one as the request's sequence number, fills its coordinates into the packet, sets the request's *exLimit* to $EXPLORE_LIMIT - 1$, and adds itself as the first entry of *visitedNodes* in the packet. Note that the first hop from a requester is selected based on the goodness heuristic (in (7)), since the gradient ascent

of the total field φ (in (2)) does not apply for positions at food nodes.[2] Maintained in the *step-one-choice table*, goodness values give higher desirabilities to neighbors through which food sources with higher food values and shorter distances could be reached. The neighbor j with the highest goodness value will be chosen, as long as j does not belong to node i's *connection table* (as shown in (8)). This exclusion avoids the reuse of successfully explored neighbors to encourage exploration in new directions for potentially unreached food nodes. Once a next hop is decided, its id is appended to the end of *visitedNodes* and the request is sent out.

Algorithm 3 shows how a FORWARD REQUEST packet is processed. When node i receives a FORWARD REQUEST packet, it updates its local *nearby-food table* by setting the requester's food value to zero, where the requester is retrieved from the first entry of *visitedNodes*. To be consistent with *nearby-food table*, the corresponding goodness value is deducted from the *step-one-choice table* as well. Note that node i updates the above tables even if the FORWARD REQUEST packet might not be addressed to itself, which takes advantage of broadcast nature of wireless communications to speed up the information propagation.

After that, node i checks if it has seen this request before, which is identified by the tuple $\langle requester, seqNo \rangle$. Node i also examines if its id matches the last entry in the request's *visitedNodes* field which designates the next hop for the packet. If this is an old request, or the packet is not addressed to itself, the request is discarded. Otherwise, node i knows that this new request packet is addressed to itself and thus accepts it. Then, node i checks if it is a food node itself. If so, node i realizes to be the destination of the request packet. If it is the first request packet received from the requester, node i becomes a replier (Line 8 of Algorithm 3).[3] The FORWARD REQUEST is then turned into a BACKWARD REPLY packet to return to the requester. In the packet format shown in Fig. 3(b), the only difference between a reply packet and its corresponding request packet is that a *replier* field replaces the *exLimit* field, and correspondingly the replier's coordinates are recorded in the packet rather than the requester's. The reply packet uses the same sequence number as the received request, and is identified by the same tuple as the request packet. The field *visitedNodes* is directly copied from the received request except that the last entry is removed, so that the reply packet could follow the reverse route back to the requester. The new last entry in *visitedNodes* is then recorded as a *parent* in node i's *connection table*, as well as the next hop for the reply packet. If this is the first time food i is connected to the network, besides sending back a BACKWARD REPLY packet, node i is also allowed to release a FORWARD REQUEST to explore other unreached food nodes, by invoking the procedure *SendRequest*() described in Algorithm 2.

If node i is not a food node, then it becomes a potential forwarder. It checks if the request's *exLimit* field is positive. If not, the request is discarded. Otherwise, node i tries to forward the request. It decreases the packet's *exLimit* by one, and invokes the procedure *GetNextByGradient*(*tabu*) (shown in Algorithm 4) to compute a next hop based on gradient ascent, where *tabu* is a set of nodes excluded for the choice of the next hop, comprising all nodes inside node i's *connection table*, as well as nodes that have already been visited by this request packet. The introduction of *tabu* encourages exploration of new directions for potentially unreached food nodes and helps raise the efficiency of exploration. Once a valid

[2]At a food node n, the displacement vector is $\mathbf{x} - \mathbf{x_n} = \mathbf{0}$. Thus the logarithm of the zero magnitude is undefined (approaching $-\infty$) in (2).

[3]Notice that re-connections by the same requester are suppressed in the protocol, while re-connections by different requesters are allowed, thus trading some path redundancy for efficiency.

Algorithm 3 Node i processing a FORWARD REQUEST

```
Input:
req ← incoming FORWARD REQUEST packet
        identified by ⟨visitedNodes[0], seqNo⟩

Begin:
Set zero food value in node i's nearby-food table:
  j ← req.visitedNodes[0]
  originVal ← nfTab_i(j).f
  nfTab_i(j).f ← 0

Decrease goodness in node i's step-one-choice table:
  b ← nfTab_i(j).n_h
  goodness_i(b) ← originVal / nfTab_i(j).d_h

  socTab_i(b) ← socTab_i(b) − goodness_i(b)

if ⟨req.visitedNodes[0], req.seqNo⟩ has not been seen before
AND i = last entry in req.visitedNodes then
  if nodeType_i = FOOD then
    if i is reached by req.visitedNodes[0] for the first time then
      Convert req to a BACKWARD REPLY rep:
        rep.replier ← i
        rep.x ← xCoord_i
        rep.y ← yCoord_i
        rep.seqNo ← req.seqNo
        rep.visitedNodes ← req.visitedNodes

      Add next-to-last entry in rep.visitedNodes
        to connTab_i.parents

      Remove last entry from rep.visitedNodes
        and send out rep
    end if
    if i is reached the first time then
      Invoke SendRequest ()
    end if
  else
    if req.exLimit > 0 then
      req.exLimit ← req.exLimit − 1
      tabu ← connTab_i ∪ req.visitedNodes
      nextHop ← invoke GetNextByGradient (tabu)

      if nextHop ≠ INVALID then
        Append nextHop to req.visitedNodes
          and send out req
      end if
    end if
  end if
end if
```

next hop is chosen, its id is appended to the end of *visitedNodes* and the request packet is rebroadcasted. Otherwise when the next hop is *INVALID*, the request packet is discarded.

Algorithm 4 Procedure *GetNextByGradient*(*tabu*) executed by node i

1: **Parameters:**
2: *tabu* ← set of nodes excluded from choice of next hop

3: **Begin:**
4: Compute the *mean* of all food values in node i's *nearby-food* table:
$\bar{f} \leftarrow \text{mean}_{j \in nfTab_i}\, nfTab_i(j).f$

5: Compute the *gradient* vector $\nabla(\nabla_x, \nabla_y)$ of the total field defined in (2):

$$\nabla_x = \sum_j \frac{2(nfTab_i(j).f - \bar{f}) \times nfTab_i(j).d_x}{nfTab_i(j).d_x^2 + nfTab_i(j).d_y^2}$$

$$\nabla_y = \sum_j \frac{2(nfTab_i(j).f - \bar{f}) \times nfTab_i(j).d_y}{nfTab_i(j).d_x^2 + nfTab_i(j).d_y^2} \tag{9}$$

6: **if** $\nabla \neq \mathbf{0}$ **then**
7: *nextHop* ← the neighbor $k(\in ngTab_i \setminus tabu)$ such that vector **ik** is the closest to vector ∇ direction
8: **else**
9: *nextHop* ← *INVALID*
10: **end if**

11: Return *nextHop*

As mentioned above, a forwarder node i invokes the procedure *GetNextByGradient*(*tabu*) to compute a next hop for a FORWARD REQUEST. Algorithm 4 shows how the calculation is done using gradient ascent of the singular potential model defined in (2). The gradient vector $\nabla(\nabla_x, \nabla_y)$ of the total field φ at current node i is obtained by calculating the partial derivatives with respect to x-axis (∇_x) and y-axis (∇_y), respectively, as shown in (9). Since vector ∇ actually denotes the precise direction which promises the maximum increase in food potentials, the next hop is thus selected to be the neighbor k with vector **ik** being closest to vector ∇ in direction, as long as k is not in the *tabu* set. Note that when every food in node i's *nearby-food table* is a *zero-food*, which means all food nodes nearby have already been connected to the network, the above computation results in a *zero-gradient* (**0**), where $\nabla_x = \nabla_y = 0$. In this case, no next hop is available and thus *INVALID* is returned, which indicates the end of the local food exploration at node i.

3.1.4 Backward explore reply

Once a food node is reached by a fresh FORWARD REQUEST, it releases a BACKWARD REPLY via the reverse path back, acknowledging the requester as well as announcing to others its connection to the network. The packet format of a BACKWARD REPLY is similar to that of a FORWARD REQUEST, as compared in Figs. 3(a) and 3(b). Specifically, line 9 of Algorithm 3 shows how a reply packet is converted from the received request packet.

Algorithm 5 Node i processing a BACKWARD REPLY

1: **Input:**
2: $rep \leftarrow$ incoming BACKWARD REPLY packet
identified by $\langle visitedNodes[0], seqNo \rangle$
3: $prev \leftarrow$ the node from which rep was received

4: **Begin:**
5: Set zero food value in node i's *nearby-food* table:
$j \leftarrow rep.replier$
$originVal \leftarrow nfTab_i(j).f$
$nfTab_i(j).f \leftarrow 0$

6: Decrease goodness in node i's *step-one-choice* table:
$b \leftarrow nfTab_i(j).n_h$
$goodness_i(b) \leftarrow \frac{originVal}{nfTab_i(j).d_h}$

$socTab_i(b) \leftarrow socTab_i(b) - goodness_i(b)$

7: **if** $i =$ last entry in $rep.visitedNodes$ **then**
8: Add $prev$ to $connTab_i.children$

9: **if** $i = rep.visitedNodes[0]$ **then**
10: Invoke *SendRequest* ()
11: **else**
12: **if** $\langle rep.visitedNodes[0], rep.seqNo \rangle$ is seen before **then**
13: Add next-to-last entry in $rep.visitedNodes$
to $connTab_i.parents$

14: Remove last entry from $rep.visitedNodes$
and send out rep
15: **end if**
16: **end if**
17: **end if**

Algorithm 5 presents the pseudo code of how node i processes a received BACKWARD REPLY packet. First, node i updates the local food information by setting the replier as *zero-food* in its *nearby-food table*, as well as deducting the corresponding goodness value from its *step-one-choice table*. Notice that we have mentioned similar steps of updating these tables when describing how a node processes an ADVERTISEMENT (lines 8–10 of Algorithm 1) and FORWARD REQUEST (lines 4–5 of Algorithm 3). This is because both request and reply packets can serve as *zero-food* advertisements.

Next, node i checks if the reply is addressed to itself by examining the last entry in the *visitedNodes* field. If its id matches, it adds the previous hop node as a *child* into its *connection table*. Moreover, if node i is the requester that the reply is destined to, it initiates another FORWARD REQUEST packet since its previous request is successfully echoed. Otherwise, node i becomes a forwarder, and checks if it has previously seen any request packet identified by the same tuple $\langle visitedNodes[0], seqNo \rangle$ as the reply packet. If yes, node i removes the last entry of *visitedNodes*, adds the new last entry as a *parent* into its *connection table*, and rebroadcasts the BACKWARD REPLY. If not, node i discards the packet since it should have seen the request packet before the corresponding reply coming back. Note that the *con-*

nection table should only be updated with a BACKWARD REPLY packet, since a FORWARD REQUEST packet might fail to reach any food node at all. As a result, the path of the FORWARD REQUEST exploration is volatile, and thus should not be recorded in the *connection table*.

The design of the ⟨*request, reply*⟩ message pair is essential for the validity and efficiency of the protocol. Each requester (food or sink) has only one *free* request to send when connected for the first time, after which it can only send a new request at the *cost* of receiving a reply echoing the previous request. Both request and reply packets also serve as *zero-food* advertisements to keep local food information up-to-date. Ideally, when all the food nodes have been connected to the network, no valid next hops for ongoing request packets (if any) are available (as described in Algorithm 4). Therefore, all request packets will be discarded, and no exploration packets will be generated any more—the protocol then terminates. However, in realistic wireless communications, the physical channel is contentious and lossy. The protocol might terminate before all food nodes are successfully connected to the network, because both request and reply packets might get corrupted or lost in the network. We will evaluate the performance of our protocol via realistic network simulations in Sect. 4.1.

We now analyze the message complexity of the protocol. Suppose there are M nodes in the network, which includes N data sources and one data sink, *FOOD_RANGE* is K hops, and the average node degree in the network is A. Then there are M 1-hop HELLO messages and NA^{K-1} K-hop ADVERTISEMENT to be sent. Next, we define *efficiency* C to be the ratio of the number of non-food relays to the number of food nodes, and *robustness* B to be the average number of parents for each food node. The number of food explore messages (including FORWARD REQUEST and BACKWARD REPLY) is then about $2NCB$, since each food needs a pair of the request/reply messages with average C hops in exploration length to be connected each time, and each food is connected B times on average.

For the speed of convergence, the path growth protocol terminates fast since food nodes are connected concurrently at an exponential rate. Specifically, starting with only the sink, if in each concurrent growth iteration every connected food nodes (including the sink) can reach out to connect one other food source, it will take $O(\log_2(NB))$ concurrent growth iterations for the protocol to connect all food sources and then terminate.

3.2 Path evolution (data sinking) protocol

We adapt the slime mold tube evolution model from maze solving to design a localized data sinking protocol for WSNs. Specifically, we map the maze exit to the single data sink in WSNs with only one data sink (as the single datum node with zero pressure), since continuity (3b) requires that all data must flow into it. We will address WSNs with multiple sinks at the end of this section. Besides, food nodes with positive net outflux (e.g., $m_i = +1$) correspond to data source sensors, and maze junctions ($m_i = 0$) correspond to other sensors as potential relays. We map tubes in slime mold to communication links in WSNs, with nutrient flux q_{ij} corresponding to the data flow from node i to next hop j. In addition, the set N_i of neighbors (in (3b)) maps onto the set of nodes within the transmission radius of node i. Thus, there is a communication link between node i and all members of N_i. For simplicity, we assume all transmission radii are identical, so that all communication links are bi-directional. While the tube length L_{ij} may be a useful tuning parameter to map onto special restrictions on network connections, we do not pursue these issues in this paper and simply assume equal L_{ij}'s throughout the network corresponding to identical 1-hop transmission range.

The key mechanism of the path evolution protocol consists of alternate cycles of local computation and state synchronization at a fixed interval of Δt_{sync}, as demonstrated in Fig. 4.

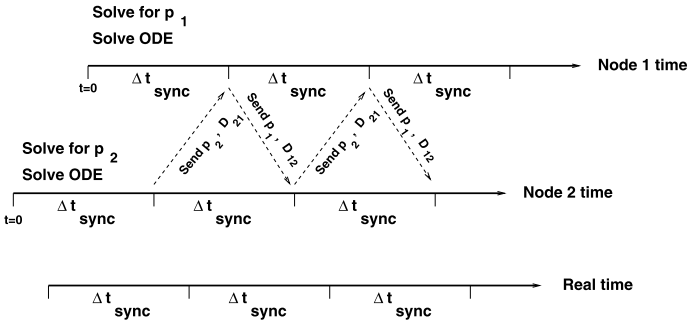


Fig. 4 Schematic diagram of local computation followed by synchronization of protocol variables. Nodes 1 and 2 perform computations independently within a fixed cycle of Δt_{sync}, and share information afterward. New pressure data from neighbors are used after the local computation cycle is completed

We initialize pressure values uniformly as one (except for the sink as zero), with D_{ij}'s randomly selected between zero and $D_{\max}$. Every node then performs iterative computations at a fixed interval of Δt_{sync} to solve (3b) for its own pressure (except at the sink) and ODE (3c) for link diameters. At the end of each Δt_{sync} cycle, a node broadcasts its latest computing results to neighbors. Meanwhile, each node keeps updating perceived neighbor pressures for use in future local computations. Notice that the localized routing protocol operates on every node in real time without explicit clock synchronization across the network. The only requirement is that clocks run forward at approximately the same rate in every node. The data sinking protocol is described in the following.

3.2.1 Local data structures

Each node i has an associated node type ($nodeType_i$), net outflux ($netFlux_i$), pressure value (p_i), and *neighbor table* ($nTab_i$).

- $nodeType_i$: denotes one of the three node types—*SINK* (the single datum node with fixed zero pressure), *SOURCE* (data source sensors), or *OTHER* (as potential relays). Note that only one node may have the node type of *SINK*, which corresponds to the data sink in a single-sink WSN.
- $netFlux_i$: specifies the pure flux m_i flowing out of node i into the network, which is positive at any data source, zero at any relay node, and unspecified/ignored at the single data sink.
- p_i: denotes node i's own pressure state. The pressure of each node other than the sink is initialized as one and iteratively solved based on (5), while the sink's pressure remains zero at all time.
- *Neighbor table* $nTab_i$: $nTab_i(j)$ denotes the *neighbor table* entry corresponding to the one-hop neighbor j of node i, with the form of $\langle p_j, D_{ij}, q_{ij}, L_{ij}\rangle$. Specifically, p_j (as neighbor j's pressure) is perceived from regular neighbor updates. The diameter D_{ij} and the flux q_{ij} of the link (i, j) record the iterative local computation results based on (6) and (3a), with D_{ij} periodically synchronized by neighbor updates. By keeping track of the local dynamics in the evolving system of (3), the *neighbor table* is used to construct the routing table for data gathering operations in WSNs. In particular, neighbor entries with outward fluxes indicate next hops leading to the sink, with the associated flux distribution further guiding either a single/best-path or a multi-path data forwarding.

Fig. 5 Format of synchronization (SYNC) packet

id	p	$\#entries$	n_1	D_{id,n_1}	$\ldots$	n_k	D_{id,n_k}

3.2.2 Local computation cycle

Between two synchronizations of pressures and link diameters with neighbors, each node i performs one local computation cycle in R rounds to solve (3) for pressure p_i (except for the sink), neighbor link flux q_{ij}'s, and diameter D_{ij}'s, based on (5), (3a), and (6), respectively. Algorithm 6 summarizes the process of local computation at node i.

Algorithm 6 Node i doing local computation

```
Begin:
for round = 1 to R do
    if nodeType_i ≠ SINK then
        Solve linear (3b) for p_i based on (5)
    end if
    for all j ∈ nTab_i do
        Compute q_ij based on (3a)
        for step = 1 to S do
            Solve ODE (3c) for D_ij based on (6)
        end for
    end for
end for
```

Each round of the local computation works as follows. First, every node i (except for the sink) calculates its pressure p_i by solving the linear equation system of Kirchhoff's law (3b), given neighbors' p_j's and D_{ij}'s based on (5), while the sink remains zero pressure as the minimum pressure level. Next, for each of its neighbor j, node i determines the corresponding flux q_{ij} with the freshly calculated p_i based on (3a). Then it solves the adaptive system of (3c) for D_{ij} based on (6) in S discrete time steps, with each step's result based on the previous one and fed into the next. Since each step signifies Δt time as shown in (6), the actual time for one cycle of local computation (including R rounds of S steps) is $T_{\text{comp}} = R \times S \times \Delta t$. Note that the synchronization interval $\Delta t_{\text{sync}} > T_{\text{comp}}$, so that a node could finish the current cycle of local computation before sharing its latest results with neighbors.

3.2.3 Synchronization cycle

Before each cycle of local computation, every node i shares its own pressure p_i and synchronizes D_{ij} with its neighbors by broadcasting a one-hop synchronization (SYNC) packet. As shown in Fig. 5, the SYNC packet contains the sender's id and its current pressure p, together with the latest *neighbor-diameter table*, which includes the total number of entries (*#entries*), followed by each entry $\langle n_j, D_{id,n_j} \rangle$. In particular, D_{id,n_j} represents the diameter of the link between the sender and its neighbor n_j, retrievable from the sender's *neighbor table*. Note that the first synchronization also serves the purpose of neighbor discovery, which contains the initial pressure and an empty *neighbor-diameter table*.

Algorithm 7 Node i processing a synchronization packet

1: **Input:**
2: $sync \leftarrow$ incoming synchronization packet

3: **Begin:**
4: $j \leftarrow sync.id$

5: **if** $j \notin nTab_i$ **then**
6: Add an entry of j to node i's *neighbor* table:
$nTab_i(j).p_j \leftarrow sync.p$
$nTab_i(j).q_{ij} \leftarrow 0.0$
$nTab_i(j).L_{ij} \leftarrow 1.0$

7: **if** $sync.\#entries == 0$ OR
$i \notin$ *neighbor-diameter* table of *sync* **then**
8: $nTab_i(j).D_{ij} \leftarrow rand(i+j)$ /* initial random value $\in [0, D_{\max}]$ shared between i and j */
9: **else**
10: $nTab_i(j).D_{ij} \leftarrow sync.D_{ji}$
11: **end if**
12: **else**
13: $nTab_i(j).p_j \leftarrow sync.p$

14: **if** $i \in$ *neighbor-diameter* table of *sync* **then**
15: $nTab_i(j).D_{ij} \leftarrow (nTab_i(j).D_{ij} + sync.D_{ji})/2$
16: **end if**
17: **end if**

Algorithm 7 presents the pseudo code of how node i processes an incoming synchronization packet. Upon receiving a SYNC packet, each node tries to synchronize its *neighbor table* in terms of neighbor pressure and link diameter based on information from the packet. In case of the first time to hear from sender j, node i will insert in its *neighbor table* a new entry for j. Specifically, D_{ij} in the new entry is either assigned as the corresponding D_{ji} (Line 10) recorded in the packet's *neighbor-diameter table* if it exists, or otherwise initialized as a random value $rand(i+j)$ shared between nodes i and j (Line 8). Notice that the latter case applies when receiving a first batch of SYNC packets which serve as neighbor discovery, and the random value of $rand(i+j)$ uniquely dependent upon $(i+j)$ ensures that $D_{ij} = D_{ji}$ at the beginning. If sender j already exists in node i's *neighbor table*, p_j is updated by the packet, and D_{ij} is synchronized as the average between its computed D_{ij} and perceived D_{ji} from the packet.

3.2.4 The issue of asymmetric neighborhoods

Due to the unreliability of wireless communications, synchronization packets may be lost or corrupted. If packets are lost while discovering neighbors, this can create the situation of "asymmetric neighborhood." For instance, node i can hear (sometimes) from node j thus recording j in its *neighbor table* and counting p_j and D_{ij} during its local computations. However, node j could not receive successfully from node i, thus neither considering node i as its neighbor nor including node i in its local computation. The problem becomes

aggravated when node i happens to calculate a large amount of outward flux q_{ij} to node j—this would be totally unexpected at node j since it is impossible for j to locally compute a corresponding inward flux q_{ji} from i. This discrepancy would potentially disrupt the global flow continuity of (3b) and render the model solving and the protocol unable to converge. Asymmetric neighborhood becomes widespread as the number of nodes within transmission range increases which causes severer channel contentions.

Although packet loss may be mitigated via MAC layer schemes like random jitters, it could never be eliminated as long as the channel is unreliable. We take an alternative approach to contain the negative effect of packet losses so as to prevent the asymmetric neighborhoods, by introducing the neighborhood threshold (*neighThresh*). Every node tracks the number of SYNC packets sent out (*#pktSent*), as well as received from each neighbor (*#pktRcvd*) recorded within each neighbor entry of its *neighbor table*. During local computation, nodes only consider "true" neighbors that satisfy

$$\#pktRcvd > \#pktSent \times neighThresh \tag{10}$$

where $neighThresh \in [0, 1]$, and needs to be carefully selected. In practice, the neighborhood threshold is usually chosen within $[0.5, 1]$—the more contentious and poorer quality the channel is, the bigger the threshold should be. The basic rationale of this approach is the assumption that the channel quality between two nodes is roughly symmetric when both nodes transmit using the same power within a short time period.

3.2.5 Data forwarding

In WSNs, the flux distribution obtained by solving the above equation (3) constitutes a routing table at each node which may be used to move data from data sources to data sink(s) following a multi-path data forwarding scheme. The outward fluxes from a node i guide how packets are forwarded through the network. If we denote O_i to be the subset of $k \in N_i$ such that $q_{ik} > 0$, O_i then represents the next-hop neighbors of node i on a route toward the sink. We define the ratio

$$w_{ij} = \frac{q_{ij}}{\sum_{k \in O_i} q_{ik}}, \quad j \in O_i. \tag{11}$$

If node i has α data packets to send, αw_{ij} packets (rounding as necessary) would be sent to node j for each $j \in O_i$. While the w_{ij}'s do not necessarily represent probabilities, we could pose a stochastic forwarding protocol by forwarding each packet arriving at node i to node j with probability w_{ij}.

3.2.6 Multi-sink scenario

In WSNs with multiple sinks, data collected by a large number of sensors must be transmitted efficiently and reliably to corresponding sink nodes. For instance, wireless sensor and actor networks (WSANs) include several actor nodes which gather, process, and act upon data from numerous sensors. From now on, we simply use WSANs to refer to multi-sink WSNs. There are two kinds of coordination in WSANs (Akyildiz and Kasimoglu 2004). First, actor–actor coordination selects the actor with which each sensor is associated. Second, sensor–actor coordination finds the optimal paths for sending and receiving information between an actor and its associated sensors.

While the slime mold tube dynamics model could be directly applied to sensor–actor coordination for finding data gathering paths, there is no straightforward analogy between

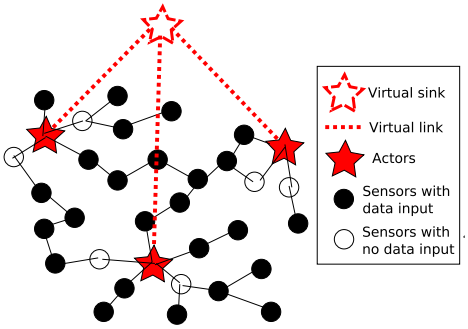


Fig. 6 A WSAN augmented by adding one "virtual sink" which is linked only to the actors

slime mold tubular network formation and actor–actor coordination for associating sensors to actors. To address both coordinations using our existing framework, we augment the WSAN by adding a "virtual sink" connected only to the designated actors as shown in Fig. 6. Consequently, we specify the virtual sink to be the single datum node with a constant baseline pressure of zero. This ensures that the virtual sink should be the only node to which all data would flow. In addition, we treat actor nodes as pure relays (neither *SOURCE* nor *SINK*) to have zero net outfluxes. Then the augmented WSAN becomes a WSN with a single (virtual) sink which fits in our existent framework. Since actors are the only neighbors of the virtual sink, all fluxes to the virtual sink must be relayed by one of these actors as the last hop in the routing path. Notice that in our earlier work on WSAN coordinations (Rossi et al. 2007), we assumed a different approach with both the advantage of building paths to several actor as a built-in feature and the disadvantage of demanding spatial location awareness. Comparatively, our current approach requires no location awareness. If we remove the virtual sink together with all virtual links forcing all fluxes to stop at the actors, the system is reduced to the original WSAN coordinations: All paths from data sources end at actors and each data source is associated with at least one actor, thus handling both sensor–actor and actor–actor coordinations simultaneously.

4 Validation and evaluation

To evaluate the performance of the two routing protocols, we conduct extensive simulation of the protocols in the QualNet simulator (Scalable Network Technologies, Inc. 2011) using realistic sensor networking scenarios. Quantitative metrics are collected to evaluate the efficiency and the robustness of the routing results, shown together with visual snapshots of the resulting routes. Moreover, for both protocols, we compare the QualNet simulation results with the exact or numerical Matlab solutions of corresponding mathematical models, so as to validate our protocols design.

4.1 Path growth protocol

We conduct extensive simulations using both grid and random network topologies. To mimic the movement of slime mold over a continuous space in a discrete manner, we first use a grid topology of discrete lattice nodes that can approximate each intermediate step in the paths. Each network scenario consists of 900 nodes placed in a grid topology over a terrain of size 870×870 m^2. Each node is positioned 30 meters away from its horizontal and vertical neighbors, and is equipped with a radio transceiver capable of transmitting signals

Table 1 Parameters for path growth protocol

FOOD_RANGE	1–10 hop
FOOD_PORTION	0.05–0.99
EXPLORE_LIMIT	15 hop
AD_TIMER	3 sec
REQUEST_TIMER	6 sec

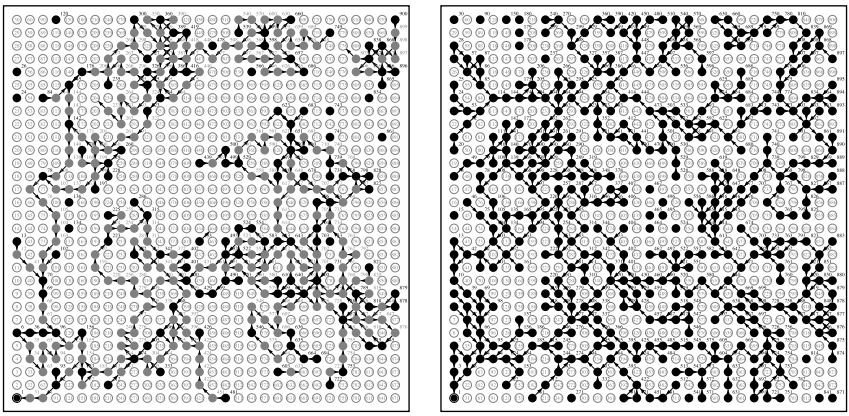
up to approximately 43 meters, so that it may only directly communicate with immediate grid neighbors in horizontal, vertical, and diagonal directions. Therefore, the average node degree D in this grid topology is approximately eight.[4] We also simulate the protocol in random topology, where each network consists of 400 nodes randomly placed in a terrain of size 1000×1000 m^2. Each node has a transmission radius of approximately 108.2 meters, so that the average node degree D in the random topology is close to 15, which is larger than that of the grid topology. Other common network and protocol parameters for both grid and random topologies are as follows. The underlying wireless channel has a data rate of 2 Mbps, using the two-ray path loss model without fading. IEEE 802.11 DCF is used as the MAC layer protocol, and IP as the network layer on top of which the protocol is run. Each network has only one sink and a varying portion of food nodes with randomly generated locations and food values. No application traffic is employed since the sole purpose is to establish routing paths between the sink and the food nodes. *FOOD_PORTION* varies from 0.05 to 0.99 with an increment of 0.05, denoting the ratio of the number of food nodes over the total number of nodes in the network. *FOOD_RANGE* varies from 1 to 10 hops with an increment of 1 hop, representing how many hops away a food node may be known to others. Table 1 summarizes the protocol parameters used in the simulations.

We devise the following performance metrics to evaluate the quality of resulting routes:

- **Food connection ratio**: indicates the fraction of food nodes that are connected to the network (i.e., reachable to/from the sink). This metric denotes the *effectiveness* of the protocol in terms of connecting food nodes, ranging from zero to one. The higher the ratio, the better the *effectiveness*.
- **Number of non-food forwarders per connected food node**: indicates the ratio of the number of non-food forwarders which act as relays between the sink and food nodes, to the number of food nodes connected to the network. This metric denotes the *efficiency* of the resulting network connectivity in terms of the average hop count distance of exploration paths. The smaller the number, the higher the *efficiency*. A value of zero indicates perfect efficiency.
- **Average number of uplinks per connected food node**: indicates the average number of choices of established upstream links a connected food node can select to reach the sink. This metric measures the *robustness* of the resulting network connectivity in terms of the average times of re-connections plus one (for the first connection). The connectivity is robust as long as this metric is greater than one. The higher the number, the better the *robustness*.

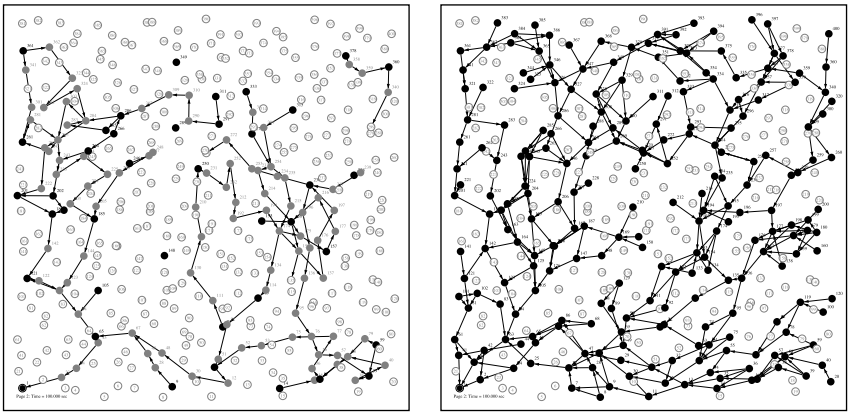
Figures 7 and 8 show the snapshots of routing results in sample networks with grid and random topology, respectively. Specifically, Fig. 7(a) presents the resulting connectivity over the 900-node grid, with a sparse *FOOD_PORTION* of 0.15 and the *FOOD_RANGE* of 3 hops. About 91.2% of the food nodes (shown in black) are connected to the sink which is

[4]Notice that only non-border nodes have eight 1-hop neighbors.



(a) (b)

Fig. 7 Sample routes in the network of grid topology, where the sink node resides in the bottom left corner. Food nodes are shown in black and non-food forwarders in gray. (**a**) *FOOD_RANGE* = 3 hops, *FOOD_PORTION* = 0.15, and (**b**) *FOOD_RANGE* = 1 hop, *FOOD_PORTION* = 0.5. (The corresponding animations are available as online supplementary material.)



(a) (b)

Fig. 8 Resulting routes in a sample network of random topology, where the sink node resides in the *bottom left corner*. Food nodes are shown in *black* and non-food forwarders in *gray*. (**a**) *FOOD_RANGE* = 3 hops, *FOOD_PORTION* = 0.1, and (**b**) *FOOD_RANGE* = 1 hop, *FOOD_PORTION* = 0.5

located at the bottom left corner. On average, each connected food node has 1.352 uplinks to the sink (robustness) and uses 1.904 non-food forwarders (shown in gray) as relays (efficiency), representing a balance between robustness and efficiency. Similarly, Fig. 8(a) shows the routing paths over a 400-node random network, with a sparse *FOOD_PORTION* of 10% and the same *FOOD_RANGE* of 3 hops. The food connection ratio is about 92.3%, with balanced robustness metric of 1.194 and efficiency metric of 2.722. One reason for the existence of some unconnected food nodes is that they are too far (more than *FOOD_RANGE* hops) away from the connected food network to be reached based on the singular potential growth model. Increasing *FOOD_RANGE* may address the problem in ideal scenarios without realistic communications. However, in realistic wireless communications, either the ADVER-

TISEMENTs initiated by some unreached food nodes might get lost during the neighbor food discovery phase, making other nodes even within *FOOD_RANGE* hops to be unaware of the food's existence, or the FORWARD REQUEST might get lost on the way to the food. Although we could mitigate packet losses via MAC layer schemes like random jitters, network layer schemes like passive acknowledgment, or a combination of both, the problem of unreliable channel cannot be easily solved completely. Moreover, increasing *FOOD_RANGE* (as well as *FOOD_PORTION*) may lead to more control packets transmitted close to each other spatially and temporally, which in turn increase the chance of packet loss from collisions. Another anomaly resulting from the unreliable channel is the "island" of intra-connected nodes which are partitioned from the main food network, an example shown at the upper right corner of Figs. 7(a) and 8(a). This is due to the loss of BACKWARD REPLY packets which prevents the establishment of the full path between a requester within the food network and its replier. However, without the knowledge of being isolated, the first-time 'connected' replier still sends out a FORWARD REQUEST to reach another unconnected food node, thus expanding the "island."

Figure 7(b) shows another sample connectivity in the grid network with a smaller *FOOD_RANGE* of one hop and a bigger *FOOD_PORTION* of 0.5. Compared to Fig. 7(a), the food connection ratio is higher (96%), with average fewer upstream links of 1.193 (still robust since the value is greater than 1) and zero non-food forwarder (perfect efficiency) per connected food. Under the same *FOOD_RANGE* and *FOOD_PORTION* values as Fig. 7(b), Fig. 8(b) shows a full 100% connection ratio with the average of 1.513 uplinks to the sink and the zero non-food forwarder in the same random network as Fig. 8(a). The better connection ratio comes from the higher *FOOD_PORTION* as well as increased average node degree (from 8 in the grid to 15 in the random topology), which means more food nodes in neighborhood, making connections of more food nodes easier with shorter exploration paths and resulting in less non-food forwarders as well. Note that the perfect efficiency of zero non-food forwarder here is determined by the one hop *FOOD_RANGE*, which means the *neighbor-food* table includes only food nodes among 1-hop neighbors. Therefore, a food node is either reached directly by a neighboring requester without any non-food relay, or remains unconnected.

While the snapshots visualize the network connectivity and qualitatively show the *effectiveness*, *efficiency*, and *robustness* of these connections, Figs. 9–12 present concise and systematic depictions of the three metrics in networks of both grid and random topologies under varying *FOOD_RANGE* and *FOOD_PORTION* quantitatively. Each measurement in the presented figures is an average of 50 different randomly generated networks for a given configuration, shown with a 90% confidence interval.

First, we vary *FOOD_RANGE* from 1 to 10 hops and compare the metrics among *FOOD_PORTION* $= 0.1, 0.3$, and 0.5 in QualNet (simulated protocol with realistic wireless communications) and *FOOD_PORTION* $= 0.1$ in Matlab (exact computations of the singular potential model in ideal scenarios with global information). Figure 9(a) shows the metric of food connection ratio in networks with grid topology. In all cases of *FOOD_PORTION*, food connection ratio increases rapidly within the initial several hops, then levels off as *FOOD_RANGE* continues to increase. This is because with the increase of *FOOD_RANGE* more food nodes are included in the *neighbor-food* table, thus exposing more targets to be reached. However, a larger *FOOD_RANGE* also introduces more packet collisions, which may offset the gain in connection ratio. Both cases of *FOOD_PORTION* $= 0.3$ and 0.5 reach their peak connection ratios at *FOOD_RANGE* $= 2$ hops, while both *FOOD_PORTION* $= 0.1$ cases (QualNet and Matlab) take more hops, since a sparse food distribution causes less packet collisions under

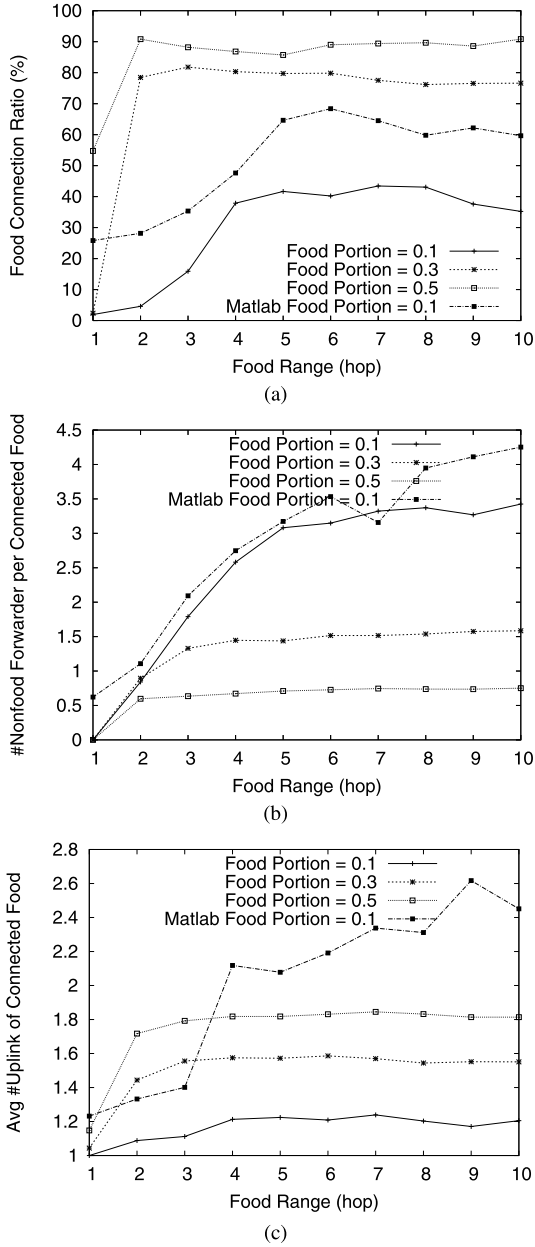


Fig. 9 In networks with grid topology, comparison among *FOOD_PORTION* $= 0.1, 0.3,$ and 0.5 in QualNet and *FOOD_PORTION* $= 0.1$ in Matlab, with *FOOD_RANGE* varying from 1 to 10 hops, in terms of (**a**) food connection ratio, (**b**) number of non-food forwarders per connected food, and (**c**) average number of uplinks of connected food

the same *FOOD_RANGE*. As observed in *FOOD_PORTION* $= 0.1, 0.3,$ and 0.5, the larger *FOOD_PORTION*, the higher the connection ratio, since more food nodes presented in neighborhood (due to a larger *FOOD_PORTION*) make themselves easier to be reached. As for the two *FOOD_PORTION* $= 0.1$ cases, the connection ratio in the WSN protocol is comparable to that of the mathematical model—about 18% less, due to the lossy communications in WSN. Comparatively, Fig. 10(a) shows a similar trend

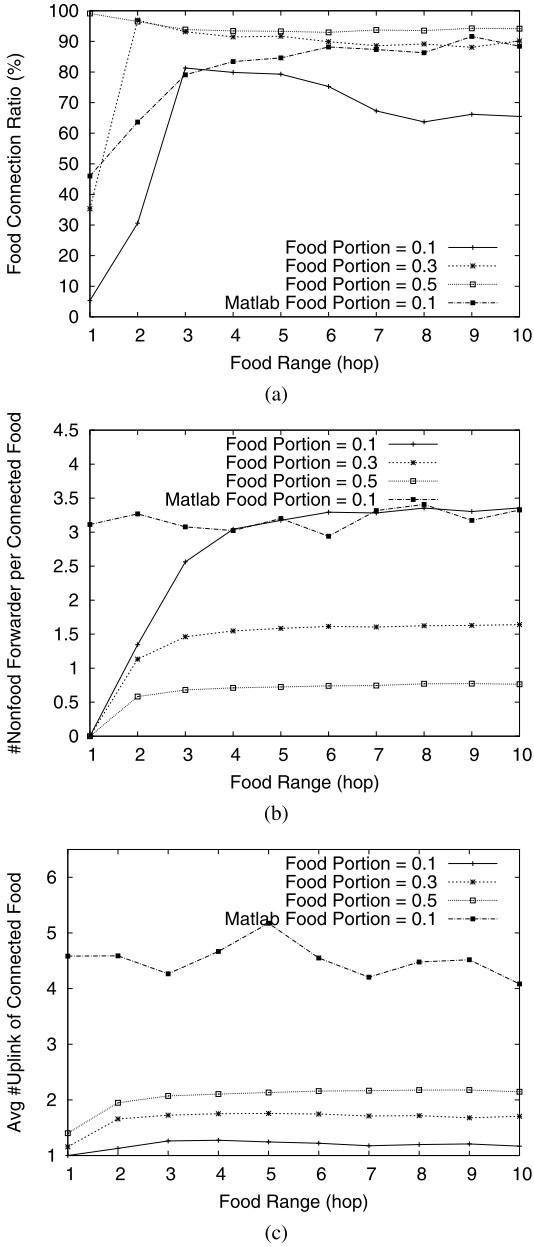


Fig. 10 In networks with random topology, comparison among *FOOD_PORTION* = 0.1, 0.3, and 0.5 in QualNet and *FOOD_PORTION* = 0.1 in Matlab, with *FOOD_RANGE* varying from 1 to 10 hops, in terms of (**a**) food connection ratio, (**b**) number of non-food forwarders per connected food, and (**c**) average number of uplinks of connected food

for the random topology, however, with higher steady connection ratios (after peak) than those in the grid topology, due to the increased average node degree (which also causes more food nodes in the neighborhood). For the same reason, it also takes less hops of *FOOD_RANGE* to reach peak connection ratios, for example, peak at *FOOD_RANGE* = 1 hop for *FOOD_PORTION* = 0.5. As for the efficiency metric, both Figs. 9(b) and 10(b) show initial increases followed by steady flats with *FOOD_RANGE*, because reaching far-

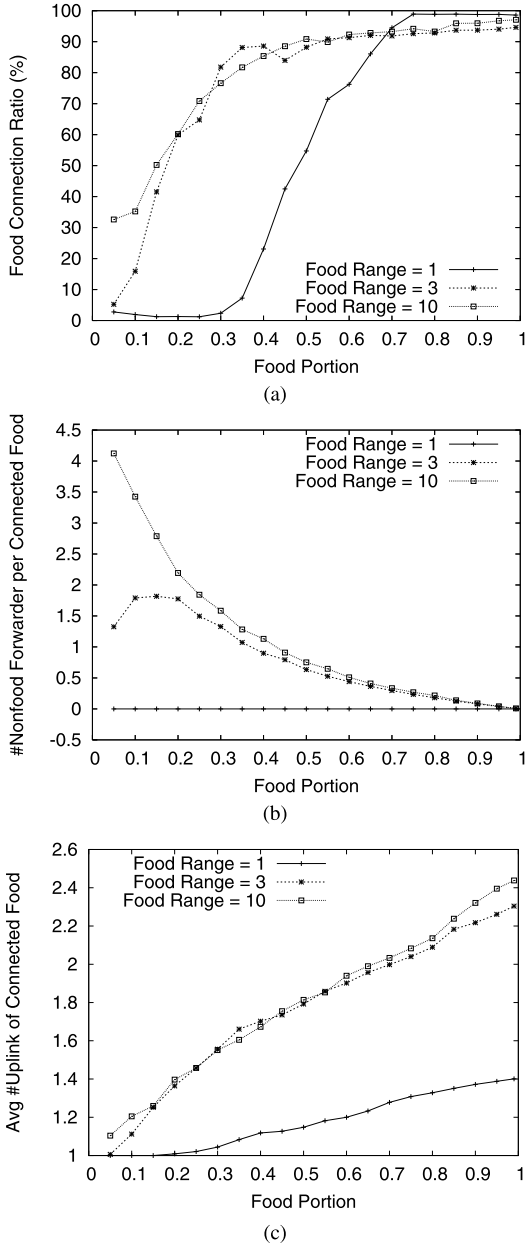


Fig. 11 In networks with grid topology, comparison among *FOOD_RANGE* = 1, 3, and 10 hops with *FOOD_PORTION* varying from 0.05 to 0.99, in terms of (**a**) food connection ratio, (**b**) number of non-food forwarders per connected food, and (**c**) average number of uplinks of connected food

away food nodes needs more relay nodes until connection ratio could no longer be increased. Among different *FOOD_PORTION* values, the higher, the better efficiency (i.e., less non-food forwarders per connected food), since more food nodes in the network (due to a larger *FOOD_PORTION*) cause shorter exploration paths between food nodes. For the two *FOOD_PORTION* = 0.1 cases, the WSN protocol is slightly more efficient in terms of forwarder number (Figs. 9(b) and 10(b)), and less robust in terms of uplink number (Figs. 9(c)

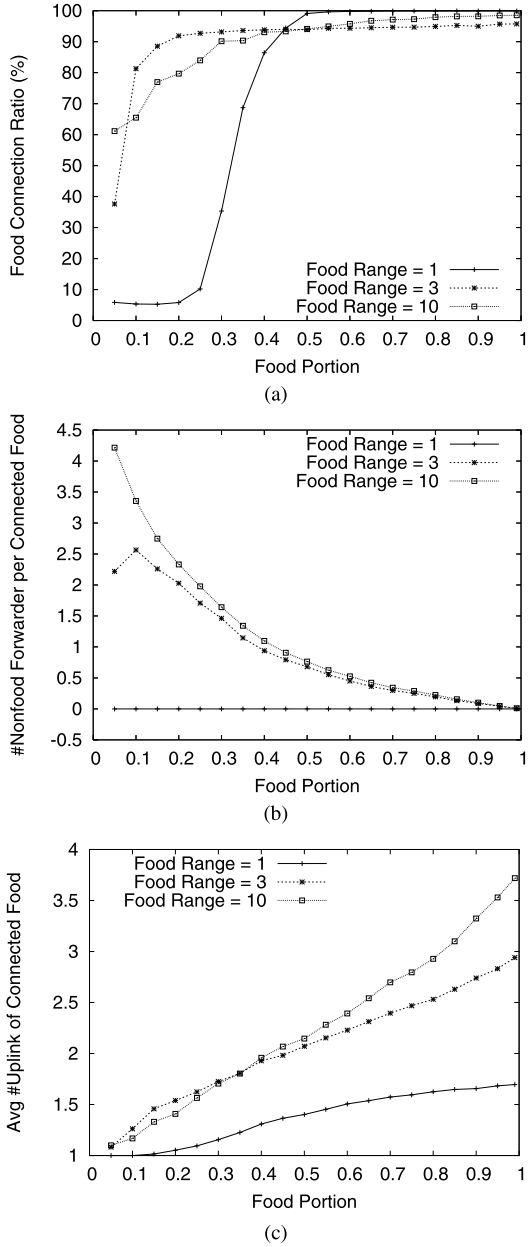


Fig. 12 In networks with random topology, comparison among *FOOD_RANGE* = 1, 3, and 10 hops with *FOOD_PORTION* varying from 0.05 to 0.99, in terms of (**a**) food connection ratio, (**b**) number of non-food forwarders per connected food, and (**c**) average number of uplinks of connected food

and 10(c)) than the singular potential model, because the protocol suppresses re-connections by same requesters (shown in Line 8 of Algorithm 3), thus trading off some robustness for efficiency. Figures 9(c) and 10(c) show that the metric of uplink number has a similar trend as the connection ratio, since a food node could be reached from more upstream food nodes when there are more potential requesters within its growing hops of *FOOD_RANGE* until the connection ratio levels off.

Second, we vary *FOOD_PORTION* from 0.05 to 0.99 and compare the same three metrics among *FOOD_RANGE* of 1, 3, and 10 hops. As *FOOD_PORTION* increases, connection ratio first builds up quickly and then levels off, as presented in Fig. 11(a) for grid topology. For *FOOD_RANGE* = 1 hop, less than 5% of food nodes are connected when *FOOD_PORTION* is less than 0.3, since each food is too far away to be known by each other in a sparse network with a small *FOOD_RANGE*. Then the ratio jumps dramatically with *FOOD_PORTION* until peaking close to 100% at *FOOD_PORTION* = 0.75 and then remains constant. Other two cases of *FOOD_RANGE* = 3 and 10 hops also peak close to 100% as *FOOD_PORTION* increases. This is because the parameter of *FOOD_PORTION* plays a major role in affecting the connection ratio, compared with the parameter of *FOOD_RANGE*. Figure 12(a) shows a similar sigmoid curve at *FOOD_RANGE* of one hop for random topology which peaks higher (100%) and earlier (at *FOOD_PORTION* = 0.55), due to increased average node degree as discussed above. Both Figs. 11(b) and 12(b) show that efficiency improves as *FOOD_PORTION* increases, until reaching the perfect case of zero non-food forwarder for both grid and random topology. Note that at *FOOD_RANGE* of one hop, the number of non-food forwarder is always zero regardless of *FOOD_PORTION*, as also evidenced in Figs. 7(b) and 9(b). Finally, the average number of uplinks increases linearly with *FOOD_PORTION* in all three cases, as seen in Figs. 11(c) and 12(c), since a denser food distribution always results in a higher food connection ratio, which produces more upstream requesters to build up the robustness.

4.2 Path evolution protocol

To validate the protocol design and the effect of adjusting parameter μ on path evolution, we present a stability analysis of a small network based on ideal numerical solutions compared with realistic protocol simulation results. Then, we evaluate the performance of the protocol on large networks under different μ values and scenarios of single/multi-sink routing and forwarding.

4.2.1 Validation with stability analysis

A thorough understanding of a simple network may give us useful insights into how parameter values affect the behavior of the whole network. We focus on a 4-node network and systematically study the stationary solutions to the tube dynamics system of (3). The 4-node network consists of one sink n_1, two sources n_2 and n_4 each with one unit of flux to be sent to the sink, and one potential relay n_3, with network layout shown in Fig. 13. We will use the following scheme for variables initialization and parameters selection in both small network validation here and large network evaluation later. Pressures p are initialized to be one at all nodes except for the sink, with the sink's pressure fixed as zero. The initial link diameters are randomly selected between zero and $D_{\max}$. We choose two representative values for the tube evolution exponent, $\mu = 0.5$ and $\mu = 2$. The net outflux is one for each food source and zero for potential relay nodes. Without loss of generality, we simply set all other parameters (a, $D_{\max}$, r, and L_{ij}) to be one in the mathematical model of (3).

Among the ideal numerical solutions, there are nine steady states for $\mu = 0.5$, and six for $\mu = 2$. We also run the protocol simulations in QualNet twenty times under each configuration with different random seeds. For $\mu = 0.5$, all trials converge to a single steady state correlating to one numerical steady state as compared in Fig. 13(a). For $\mu = 2$, the protocol reaches different steady states, each correlating to one of the six numerical steady states, with one state compared between the numerical and simulation results in Fig. 13(b). As seen

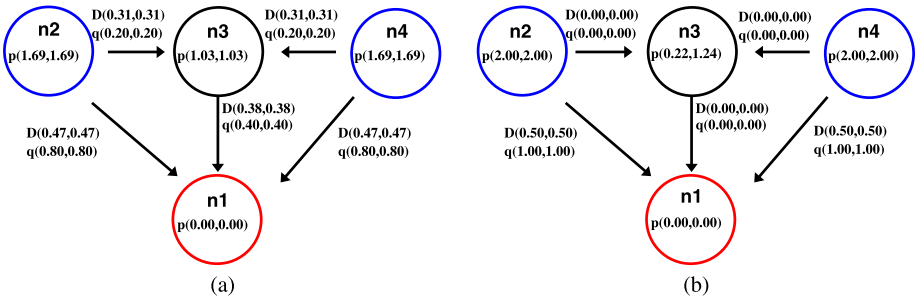


Fig. 13 Topology of a 4-node network, with comparison between steady states from numerical solutions (*left* values inside parentheses) and QualNet simulations (*right* values) under different μ values. Specifically, p, D and q represent node pressure, link diameter, and link flux, respectively, with arrows denoting flux directions. (**a**) $\mu = 0.5$, (**b**) $\mu = 2$

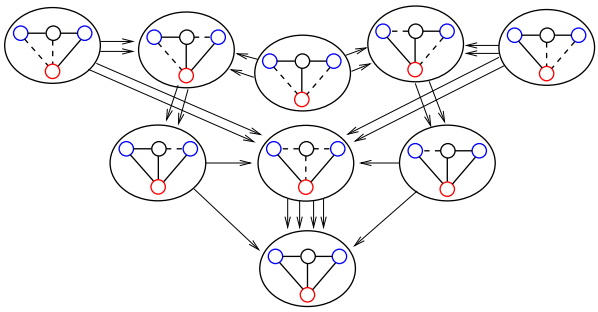


Fig. 14 Stability diagram for the 4-node network with $\mu = 0.5$. The topology of each of these nine steady solutions is shown inside the oval, with *solid lines* indicating links with $D_{ij} \neq 0$ and *dotted lines* for links with $D_{ij} = 0$. Each pair of *arrows* represents the growth of a small positive and negative amplitude perturbation along an eigenvector

in Fig. 13, the realistic simulation results concur with the ideal numerical solutions in link flux q_{ij} and link diameter D_{ij} for both μ values, which validate the protocol design. The two solutions also agree on almost all pressure p values except for node n_3 at $\mu = 2$, where both sources n_2 and n_4 send all fluxes directly to the sink n_1 without passing through n_3. Therefore n_3 has the "freedom" of choosing any pressure value as long as it does not participate in the current data gathering session. Notice that this conditional flexibility in p will not impact the protocol because it only happens at nodes with no flux flowing in or out. From a mathematical point of view, this case corresponds to the existence of a nontrivial, homogeneous solution when we solve (3a) and (3b) for the pressure.

To understand the stability of solutions to (3), we analyze the solutions of the nonlinear system of $\mathcal{F}(D_{ij}) = g(|q_{ij}|) - rD_{ij} = 0$, derived by setting $\frac{dD_{ij}}{dt} = 0$ in (3). The stationary solution to $\mathcal{F}(D_{ij}) = 0$ is defined as an implicit nonlinear system, where the implicit part (q_{ij}) is obtained by solving a nested linear equation system for pressure values using (5). Next, we compute the Jacobian matrix $[\frac{\partial \mathcal{F}_{ij}}{\partial D_{kl}}]$ evaluated at each stationary solution. The eigenvalues and eigenvectors of each matrix then determine the stability of the corresponding solution. If all eigenvalues are negative, the solution is a stable steady state. Otherwise, when there are any positive eigenvalues, the solution is unstable, which will evolve to a different steady state, with the linear evolution described by the eigenvector corresponding to the positive eigenvalue.

By analyzing the stability of the solutions to the 4-node network, we verify that parameter μ dominates the evolution of network topologies. Similar to ant-based point-to-point routing

schemes, values of μ that are above a threshold that is greater than one ($\mu > C_1 > 1$) drive the system toward configurations with single routes. Values of μ that are less than a threshold that is less than one ($0 < \mu < C_0 < 1$) drive the system toward configurations with multiple routes. The nonlinear dynamics for ant-based routing is explored in detail in Torres et al. (2010). The specific values of C_0 and C_1 vary from problem to problem. While a general proof for large networks is beyond the scope of this paper, the universal structure observed in small and large networks is the following: For small values of μ ($0 < \mu < C_0$), solutions where $D_{ij} > 0$ for all routes between the sources and the sink are stable, and other solutions are unstable. For instance, in our 4-node example, there are four possible routes between the data sources and the sink. For small μ, the solution with all four routes open is stable. Other solutions are unstable. If we increase μ so that $C_0 < \mu < C_1$, we move through a transition region where the four route solution is unstable but the three route solutions become stable. For larger values of μ, $\mu > C_1$, only the two route solution is stable corresponding to a single route connecting each data source to the sink. For the 4-node example, we choose representative values of $\mu = 0.5$ and $\mu = 2$ to explore the nonlinear dynamics of the system. We build the stability diagram over all nine solutions for $\mu = 0.5$ (including both stable and unstable numerical solutions) shown in Fig. 14, where the perturbations are computed using the eigenvectors related to the positives eigenvalues. We find that for $\mu = 0.5$ the initialization does not affect the final stable state at all, which should always be the all-link solution illustrated by Fig. 13(a). This mathematical analysis also explains why all QualNet simulations converge to this stable solution when $\mu = 0.5$. We find that these general properties transfer to larger networks.

4.2.2 Performance evaluation

To evaluate the performance of the path evolution protocol with different μ values, we conduct QualNet simulations under four different scenarios based on the combination of single/multi-sink and routing/forwarding.[5] Specifically, we simulate 100 and 300-node WSNs with varying numbers of source sensors and data sinks. All nodes are randomly distributed over a terrain of size 1000×1000 m^2. Each node in the network is equipped with a radio transceiver capable of transmitting signals up to approximately 80 meters. The underlying wireless channel has a data rate of 2 Mbps, using the two-ray path loss model without fading. IEEE 802.11 DCF is used as the MAC layer protocol, and IP is used as the network layer on top of which our protocol runs. For all simulations, we use the following variables initialization and parameters selection scheme in the path evolution protocol. Pressures p are initialized to be one at all nodes except for the sink, with the sink's pressure fixed as zero, and link diameters are randomly initialized between zero and $D_{\max}$. We choose two values for the evolution parameter μ—0.5 for $0 < \mu < 1$ and 2 for $\mu > 1$. The net outflux is one for each food source and zero for potential relay nodes. Without loss of generality, we simply set all other parameters (a, $D_{\max}$, r, and L_{ij}) to be one in the mathematical model (3). The neighborhood threshold is 0.8 to avoid neighborhood asymmetry. For each cycle of local computation, we use the number of rounds $R = 10$, the number of steps in each round $S = 1$, the step time $\Delta t = 0.001$ sec, and synchronization interval $\Delta t_{\text{sync}} = 4.01$ sec.

In the first set of simulations, there are only one sink and varying numbers of sources in each network. No application traffic is employed since the sole purpose is to connect source nodes to the data sink for routing. The following three quantitative metrics are used

[5]Notice that routing is fundamentally different from forwarding. The former computes the routing tables without dealing with data traffic, and the latter consults the routing tables to move data traffic.

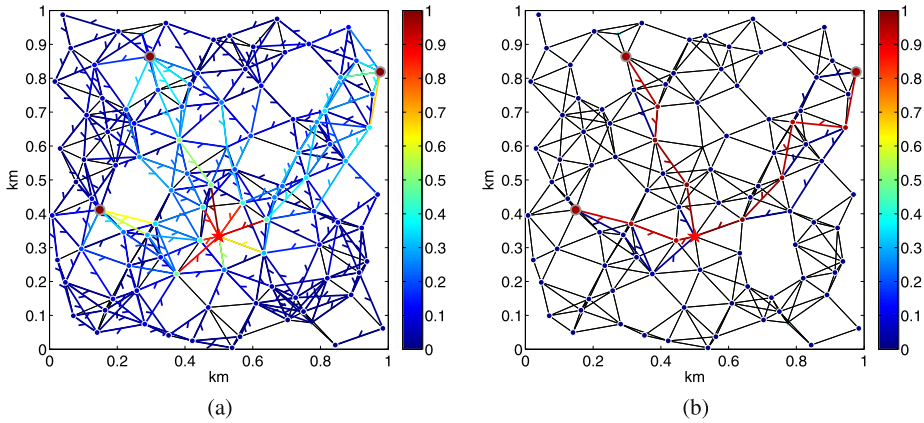


Fig. 15 Steady states obtained by QualNet for a 100-node network, including one sink (the *star* near the center) and 3 sources (large nodes with *gray circles* around them) under (**a**) $\mu = 0.5$ and (**b**) $\mu = 2$

to measure the quality of the resulting routing connectivity under $\mu = 0.5$ and $\mu = 2$ with different portions of data sources:

– *Expected hop count:* indicates the *efficiency* of the connectivity in terms of hop count distance between a source and the sink. The smaller the hop count, the higher the efficiency. Let $W = [w_{ij}]^T$ be the Markov transition matrix with w_{ij} defined in (11). Let $\mathbf{e}_l$ be a unit vector consisting of all zeros except at element l. Then $W\mathbf{e}_l$ is a vector whose jth element $(W\mathbf{e}_l)[j]$ denotes the probability of a data packet starting from node l and arriving at node j in one hop. Similarly, $(W^k\mathbf{e}_l)[j]$ represents the probability that a packet moves from node l to node j in k hops.[6] Therefore, the expected hop count from a source node l to the sink node s is

$$E(l,s) = \sum_{k=1}^{\infty} k \cdot \left(W^k \mathbf{e}_l\right)[s]. \tag{12}$$

– *Fault tolerance:* indicates the *robustness* of the connectivity. We define the γ-fault tolerance to be the probability that randomly removing γ relay nodes will *not* yet result in any source being disconnected from the sink. The higher the probability, the more robust the network.
– *Degree distribution:* defines $P(A)$ as the probability of a node in the network having a degree of A. The presence of a small number of high degree nodes may indicate a greater dependence on a few "gateway nodes" for network connectivity.

Figure 15 shows snapshots of steady state connectivity under $\mu = 0.5$ and $\mu = 2$ over a 100-node randomly generated network with one data sink and three data sources. Figure 16 depicts a 300-node network with one data sink and thirty data sources. In both figures, the width and the color of a link indicate the value of the corresponding flux normalized by the maximum flux among all links, with only links of $|q_{ij}| > 0.01$ plotted, and the arrow of each link shows the direction of the flux. The color of each node represents the amount of flux leaving that node, normalized by the maximum data outflux among all nodes. We find

[6] $W^k \equiv \prod_{i=1}^{k} W$ denotes the kth power of the square matrix W.

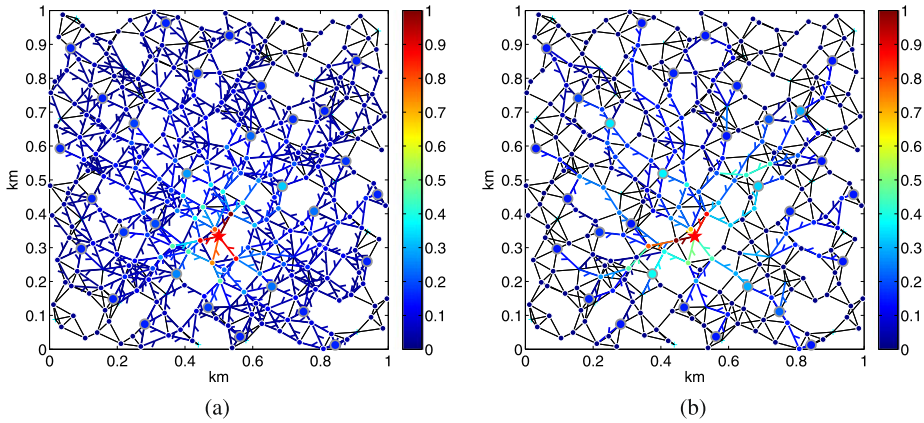


Fig. 16 Steady states obtained by QualNet for a 300 node network, including one sink (the *star* near the center) and 30 sources (large nodes with *gray circles* around them) under (**a**) $\mu = 0.5$ and (**b**) $\mu = 2$

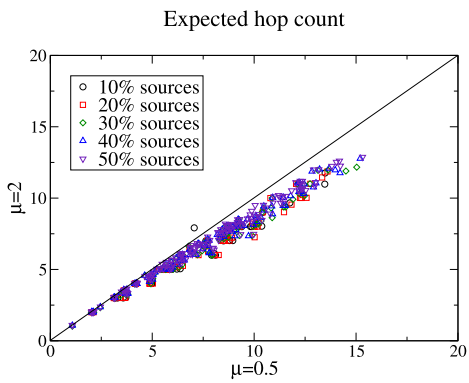


Fig. 17 Expected hop count for a 300-node network with one sink and 10% to 50% sources. The x- and y-coordinates of each data point denote the expected hop count from one source to the sink for $\mu = 0.5$ and $\mu = 2$, respectively

Table 2 Fault tolerance of a 300-node network with 1 sink and 10% to 50% sources under $\mu = 0.5$ and $\mu = 2$

Source	10%		20%		30%		40%		50%	
μ	0.5	2	0.5	2	0.5	2	0.5	2	0.5	2
1-fault	1	.9851	.9916	.9874	.9856	.9761	.9832	.9609	.9799	.9530
2-fault	.9998	.9700	.9826	.9739	.9709	.9616	.9659	.9222	.9595	.9070

that the network with $\mu < 1$ (e.g., $\mu = 0.5$) evolves toward multi-route robust connectivity, whereas the network with $\mu > 1$ (e.g., $\mu = 2$) evolves into single-route efficient topology. This confirms that the exponent μ is a critical parameter to balance efficiency and robustness by determining whether a single route or multiple routes should be created between sources and the sink.

Figure 17 demonstrates the route's efficiency in terms of expected hop count as defined in (12) for both $\mu = 0.5$ and $\mu = 2$, in a sample 300-node network with one sink and 10% to 50% of data sources. Each data point corresponds to the expected hop count values for

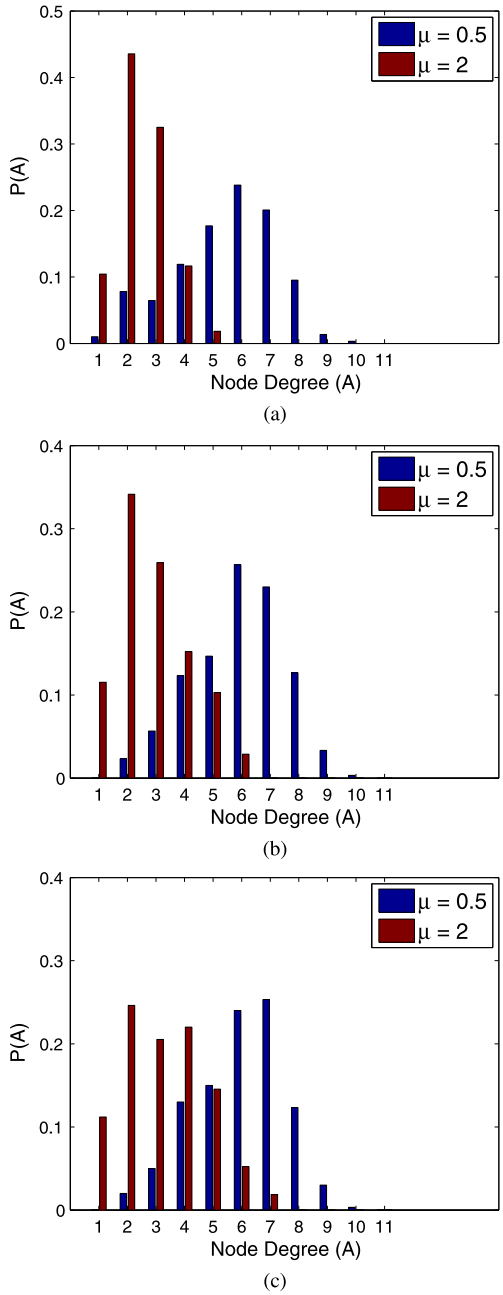


Fig. 18 Distribution of node degree for (**a**) 10%, (**b**) 30%, and (**c**) 50% sources on a 300-node network with one sink, with *bars* of $\mu = 0.5$ shown on the *left* of corresponding *bars* of $\mu = 2$

$\mu = 0.5$ (x-coordinate) and $\mu = 2$ (y-coordinate) from one of the sources to the sink. Although all points are distributed close to the diagonal, most lie below it toward the x-axis signifying longer routes from most sources to the sink with $\mu = 0.5$ than with $\mu = 2$. Linear regression of the above data shows that $\mu = 0.5$ requires approximately 10% more hops than $\mu = 2$, implying that the robustness provided by $\mu = 0.5$ comes at the cost of increased

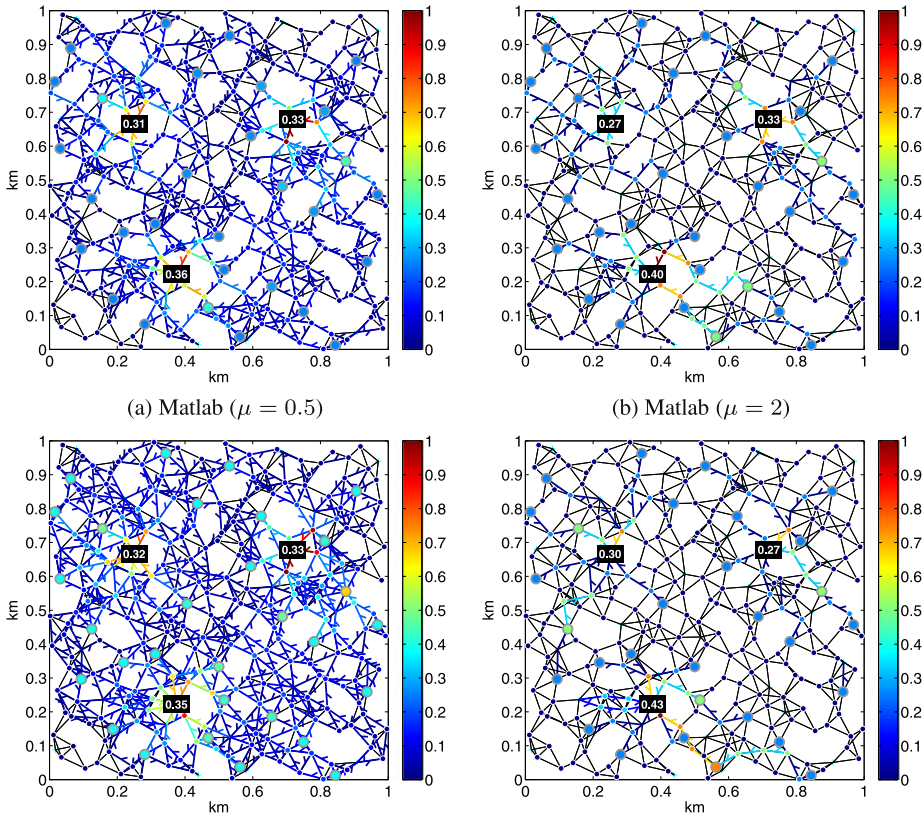
hop counts. Table 2 presents the 1- and 2-fault tolerances for the sample 300-node network with one sink and 10% to 50% of data sources, showing that $\mu = 0.5$ achieves better robustness against random node failures than $\mu = 2$. The improvement in fault tolerance for $\mu = 0.5$ corresponds to the presence of multiple paths connecting each source to the sink. In general, a multi-node failure causes a higher probability of disconnection than a single-node failure. As the network becomes more crowded with sources, it is less robust against node failures. Figure 18 compares the distribution of node degree between $\mu = 0.5$ and $\mu = 2$ in routing topologies for the same 300-node network with one sink and 10%, 30%, and 50% data sources. As expected, $\mu = 0.5$ (shown as the left bar at each node degree) leads to a higher proportion of high degree (e.g., degree $A = 6$) nodes in a multi-route connectivity, whereas $\mu = 2$ (shown as the right bar at each node degree) leads to a greater proportion of low degree (e.g., degree $A = 2$) nodes in a single-route topology. The distinction blurs slightly as more sources join the network.

In the second simulation, we demonstrate and validate our routing scheme with multiple sinks introduced in Sect. 3.2.6. Specifically, we simulate a 300-node WSAN with 30 sources and three actors spread among the sources. Again, for the sole purpose of routing paths establishment, no application traffic is employed. As discussed before, we add the "virtual sink" and connect it to each of the three actors via a dedicated point-to-point *wired* link to ensure these actors are the only neighbors of the virtual sink. Notice that in the augmented 301-node network, each sensor node bears only one interface (which is wireless), while the actors have two (one wireless interface from the original WSAN, and one added wired interface to link to the virtual sink), and the virtual sink has three wired interfaces solely connected to the actors.

Figure 19 shows the steady states for the 300-node WSAN obtained by Matlab with ideal numerical computation and QualNet with realistic communications. As expected, for both Matlab and QualNet solutions, the $\mu = 0.5$ system yields multiple routes between sources and actors, while the $\mu = 2$ system favors single routes. In addition to the above sensor–actor coordinations, the total fluxes from all sources are distributed among the three actors, with each actor's influx share biased by its location with regards to sources, which validates the effectiveness of actor–actor coordinations. The routing results of the QualNet simulations may differ from those of Matlab computation due to the unreliable wireless channel and the probabilistic nature of wireless communications, especially in large networks. However, for $\mu = 0.5$ where the only final stable state is the all-link solution, the flux distribution among three actors in the QualNet solution is quite close to that in Matlab calculation, which validates the protocol. Notice that the flux allocation among actors may be quite different between Matlab and QualNet solutions when $\mu = 2$, due to the existence of multiple steady states with different states possibly reached by each computation.

In the third set of simulations, we employ the application traffic of CBR (Constant Bit Rate) to evaluate the protocol performance for data forwarding in a 300-node WSN with a single sink under $\mu = 0.5$ and $\mu = 2$. First, we restrict the traffic to come from only one source in order to study the different effects in data forwarding with single-route ($\mu = 2$) and multi-route ($\mu = 0.5$) topology. Notice that this restriction avoids the inter-source traffic interferences—especially the funneling effect at the single sink.[7] We examine the delivery ratio and end-to-end delay under different packet generation intervals. A small packet generation interval causes more packets to be sent per time unit, resulting in more MAC layer

[7] Funneling effect refers to a product of the distinctive many-to-one, hop-by-hop traffic pattern found in sensor networks, which results in a significant increase in transit traffic intensity, collision, congestion, and packet loss as data move closer toward the sink (Ahn et al. 2006).



(a) Matlab ($\mu = 0.5$) (b) Matlab ($\mu = 2$)

(c) QualNet ($\mu = 0.5$) (d) QualNet ($\mu = 2$)

Fig. 19 Steady states for a 300-node WSAN with 3 actors (the *black rectangles*) and 30 sources (the *bold circles*) obtained by Matlab under (**a**) $\mu = 0.5$, (**b**) $\mu = 2$, and by QualNet under (**c**) $\mu = 0.5$, (**d**) $\mu = 2$. The number on each actor shows the proportion of the total fluxes from data sources flowing into that specific actor

contentions. Figure 20 shows that multi-route topologies ($\mu = 0.5$) have high delivery ratios and similar end-to-end delays compared with single-route connectivities ($\mu = 2$) in low contention scenarios (e.g., with large packet generation interval of 50 ms and above), due to the robustness of multiple routes. On the other hand, in high contention scenarios (e.g., with packet interval of less than 40 ms), multiple paths ($\mu = 0.5$) may magnify the interference, which leads to much lower delivery ratio and higher end-to-end delay.

Next, with varying number of data sources in the 300-node WSN, we allow all sources to send CBR traffic to the single sink using the same packet generation interval, but with different start times (30 ms apart) to mitigate contention. In Fig. 21, the lower two curves represent the delivery ratios in this single-sink scenario. As expected, the delivery ratio with $\mu = 2$ is 20% higher than the multi-route scheme $\mu = 0.5$ initially with 30 concurrent CBR sources. Then both ratios decrease dramatically to the same low level (below 5%) with more concurrent CBR traffic added, mainly due to the funneling effect experienced at the single sink. The reason is obvious when we compare delivery ratios between the single-sink and the multiple-sink scenarios in Fig. 21.

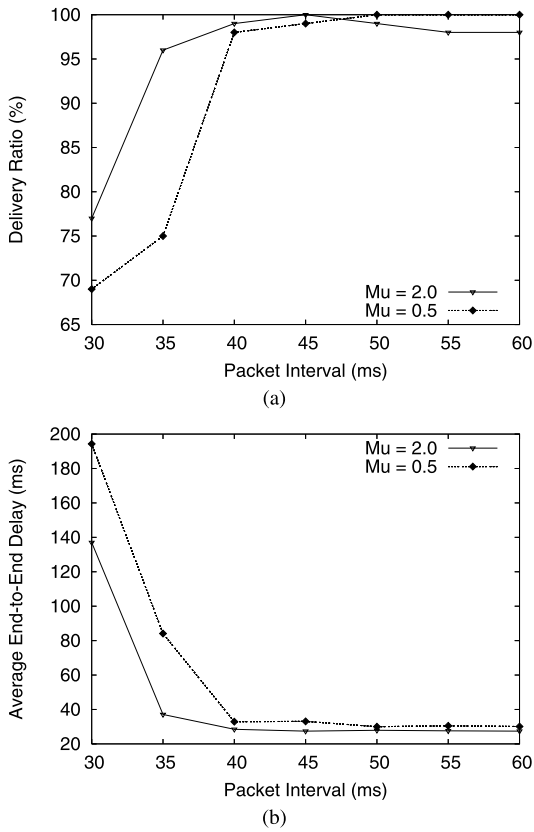


Fig. 20 (**a**) Delivery ratio and (**b**) average end-to-end delay of CBR traffic from one source to the single sink on the 300-node network, with varying packet generation intervals

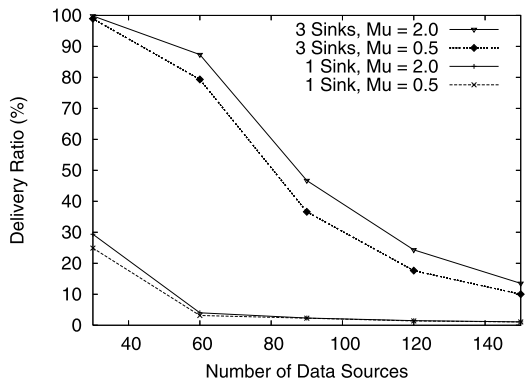


Fig. 21 Delivery ratio of CBR traffic from multiple (30 to 150) concurrent sources to either 1 or 3 sinks on the 300-node network, with the same packet generation interval and different sending time among different data sources

In the last set of simulations, we explore multi-sink data forwarding in a 300-node WSAN with three actors, and vary the number of concurrent CBR traffic sources. The upper two curves in Fig. 21 compare the data delivery ratios in single-route ($\mu = 2$) and multi-route ($\mu = 0.5$) topologies, with actors serving as the sinks. Since multiple concurrent CBR traffic causes higher contention, single-route connectivities ($\mu = 2$) have strictly better delivery ratios than multi-route topologies ($\mu = 0.5$), as evidenced before. For $\mu = 0.5$, although there

is a benefit for fault tolerance, multiple routes are more intertwined which leads to more contention. For $\mu = 2$, the system favors single routes. Although less tolerant to faults, there is less contention because routes are more segregated from one another. When comparing the upper two curves with the lower two curves, the multiple-sink scenarios work much better in delivery ratio than single-sink scenarios, due to the fact that the traffic is load-balanced among different actors through actor–actor coordination.

5 Conclusion and future work

In this paper, we have presented two slime mold inspired, self-organizing routing protocols for WSNs that take both efficiency and robustness into account. The path growth protocol treats data sources and sinks as singular potentials to establish routes from the data sinks to all the data sources. Simulation results show that the protocol is effective in establishing network connectivities, with a balance of efficiency and robustness. The path evolution protocol iteratively solves a dynamic feedback system to evolve network connectivities based on resulting flux distributions, which may further guide single/best-path or multi-path data forwarding. Through both linear stability analysis on a small network and large network simulation, we have shown that the adaptation parameter μ controls whether single-route efficient ($\mu > 1$) or multi-path robust ($\mu < 1$) connections emerge. Finally, we have conducted extensive simulations to validate and evaluate the protocol for both routing and forwarding in networks with single and multiple sinks.

Our future research follows three directions. First, in the path growth protocol, the losses of control packets both contribute to the failure of connecting certain food nodes and lead to "islands" of food nodes disconnected from the sink. We plan to investigate cross-layer design and reliable data transfer mechanisms to improve the food connection ratio and to eliminate partitioned "islands." Such improvement may also be applied to the path evolution protocol to alleviate the issue of asymmetric neighborhoods. Second, in the path evolution protocol, a uniform tube length L_{ij} is assumed throughout the network. We plan to make tube length L_{ij} a tunable parameter to represent different connection characteristics, such as link quality, transmission power, traffic load, and security. Finally, to further quantify the merits of our approach, we plan to analyze the performance bounds and convergence properties of the two protocols, and to compare, via network simulation, with existing state-of-the-art WSN protocols under various failure scenarios.

Acknowledgements The authors thank reviewers for their insightful comments. This work was supported in part by the National Science Foundation under grant CCF-0726556.

References

Adler, J., & Tso, W. W. (1974). "Decision"-making in bacteria: chemotactic response of escherichia coli to conflicting stimuli. *Science*, *184*(4143), 1292–1294. doi:10.1126/science.184.4143.1292. http://www.sciencemag.org/content/184/4143/1292.abstract. http://www.sciencemag.org/content/184/4143/1292.full.pdf.

Ahn, G. S., Hong, S. G., Miluzzo, E., Campbell, A. T., & Cuomo, F. (2006). Funneling-MAC: a localized, sink-oriented MAC for boosting fidelity in sensor networks. In *Proceedings of the fourth ACM international conference on embedded networked sensor systems (SENSYS)* (pp. 293–306). New York: ACM.

Akyildiz, I. F., & Kasimoglu, I. H. (2004). Wireless sensor and actor networks: research challenges. *Ad Hoc Networks*, *2*(4), 351–367.

Ben-Jacob, E., & Cohen, I. (1998). Cooperative organization of bacterial colonies: from genotype to morphotype. *Annual Review of Microbiology*, *52*, 779–806.

Conolly, B., Parthasarathy, P., & Dharmaraja, S. (1997). A chemical queue. *The Mathematical Scientist*, *22*, 83–91.

Culpepper, B. J., Dung, L., & Moh, M. (2004). Design and analysis of hybrid indirect transmissions (hit) for data gathering in wireless micro sensor networks. *Mobile Computing and Communications Review*, *8*(1), 61–83. doi:http://doi.acm.org/10.1145/980159.980169.

Dorigo, M., & Stutzle, T. (2004). *Ant colony optimization*. Cambridge: MIT Press.

Ducatelle, F., Di Caro, G., & Gambardella, L. (2010). Principles and applications of swarm intelligence for adaptive routing in telecommunications networks. *Swarm Intelligence*, *4*, 173–198. doi:10.1007/s11721-010-0040-x.

Fang, Q., Gao, J., Guibas, L. J., Silva, V., & Zhang, L. (2005). Glider: gradient landmark-based distributed routing for sensor networks. In *Proceedings of the 24th annual IEEE international conference on computer communications (INFOCOM)* (pp. 339–350).

Gunji, Y. P., Shirakawa, T., Niizato, T., & Haruna, T. (2008). Minimal model of a cell connecting amoebic motion and adaptive transport networks. *Journal of Theoretical Biology*, *253*(4), 659–667. doi:10.1016/j.jtbi.2008.04.017. http://www.ncbi.nlm.nih.gov/pubmed/18547591.

Heinzelman, W. R., Chandrakasan, A., & Balakrishnan, H. (2000). Energy-efficient communication protocol for wireless microsensor networks. In *Proceedings of the 33rd Hawaii international conference on system sciences* (pp. 1–10). New York: IEEE Press.

Hofer, T., & Maini, P. K. (1997). Streaming instability of slime mold amoebae: an analytical model. *Physical Review. E, Statistical Physics, Plasmas, Fluids, and Related Interdisciplinary Topics*, *56*, 2074–2080.

Intanagonwiwat, C., Govindan, R., Estrin, D., Heidemann, J., & Silva, F. (2003). Directed diffusion for wireless sensor networking. *IEEE/ACM Transactions on Networking*, *11*(1), 2–16. doi:10.1109/TNET.2002.808417.

John, F. (1986). *Partial differential equations*. New York: Springer.

Kobayashi, R., Tero, A., & Nakagaki, T. (2006). Mathematical model for rhythmic protoplasmic movement in the true slime mold. *Journal of Mathematical Biology*, *53*(2), 273–286.

Nakagaki, T., Yamada, H., & Toth, A. (2000). Intelligence: maze-solving by an amoeboid organism. *Nature*, *407*(6803), 470.

Nakagaki, T., Yamada, H., & Toth, A. (2001). Path finding by tube morphogenesis in an amoeboid organism. *Biophysical Chemistry*, *92*(1–2), 47–52.

Nakagaki, T., Kobayashi, R., Nishiura, Y., & Ueda, T. (2004a). Obtaining multiple separate food sources: behavioural intelligence in the physarum plasmodium. *Proceedings of the Royal Society of London. Series B, Biological Sciences*, *271*(1554), 2305–2310.

Nakagaki, T., Yamada, H., & Hara, M. (2004b). Smart network solutions in an amoeboid organism. *Biophysical Chemistry*, *107*(1), 1–5.

Rossi, L. F., Li, K., & Yackoski, J., Shen, C. C. (2007). Slime mold inspired coordinations for wireless sensor and actor networks. In *First ACM workshop on sensor and actor networks (SANET)* (pp. 55–56). New York: ACM.

Saleem, M., Di Caro, G. A., & Farooq, M. (2010). Swarm intelligence based routing protocol for wireless sensor networks: survey and future directions. Inf. Sci. 1–28. doi:10.1016/j.ins.2010.07.005.

Scalable Network Technologies, Inc. (2011). QualNet Simulator. http://www.scalable-networks.com.

Stewart, P. A. (1964). The organization of movement in slime mold plasmodia. In R. D. Allen & N. Kamiya (Eds.), *Primitive motile systems in cell biology* (pp. 69–78). Maryland Heights: Academic Press.

Tero, A., Kobayashi, R., & Nakagaki, T. (2007). A mathematical model for adaptive transport network in path finding by true slime mold. *Journal of Theoretical Biology*, *244*, 553–564.

Tero, A., Takagi, S., Saigusa, T., Ito, K., Bebber, B. P., Fricker, M. D., Yumiki, K., Kobayashi, R., & Nakagaki, T. (2010). Rules for biologically inspired adaptive network design. *Science*, *327*, 439–442.

Torres, C. E., Rossi, L. F., Keffer, J., Li, K., & Shen, C. C. (2010). Modeling, analysis and simulation of ant-based network routing protocols. *Swarm Intelligence*, *4*(3), 221–244.